

Trustworthy Software Engineering

-Introduction, Research Topics, and Research Techniques-

Shaoying Liu (刘少英)

Software Engineering Institute

East China Normal University



Email: syliu@sei.ecnu.edu.cn

HP: <https://home.hiroshima-u.ac.jp/sliu/>

Overview

1. Basic Concepts
2. Technologies for Trustworthy Software Engineering
3. Research Topics
4. Research Techniques

1. Basic Concepts

1.1 Software

1.2 Focus of Software

1.3 Software Engineering

1.4 Dependable Software

1.5 Dependable Software Engineering

1.1 Software

General view:

Software = Data + Program

Software engineering view:

Software = Documentation + Data + Program

What is Data (数据)?

7

1

8

Data can be **numeric** (123), **textual** ("apple"), **visual** (an image), **audio** (a sound recording), or even **binary** (010101).

What is Information?

7 = 7小时的学习

1 = 1小时的运动

8 = 8小时的学习

Information is data with meaning (信息是具有意义的数据)

What is Knowledge?

$$7 + 1 > 8$$

每天学习7小时, 运动1小时, 其学习效果大于每天只学习8小时而不运动.

Knowledge represents relations between information. (知识是信息之间的关系的表示)

What is Program?

Program = Data Structure + Algorithm

What is Algorithm?

Algorithm = a sequence of instructions or
commands

Algorithms can be used to describe sequential actions and to provide required behaviors or services.

Examples:

(1) An algorithm for morning activities:

Get up at 6am;

Drink a little boiled water;

Do physical exercise for 30 minutes;

Have breakfast;

Leave home and walk to the railway station;

Take train to the university;

Start work

(2) An algorithm for calculating the average score of a class:

1. Input all the **scores** for a given class;
2. Add up all the **scores** to get a **total score**;
3. Divide the **total score** by the **total number** of students in the class;
4. Output the **result of the division** as the average score

What is Program Code or Code?

Code = executable program on computer

Examples:

Programs written in Java, C, C++, C#. And Python are all called code.

Question for discussion:

What is the main factor that makes code difficult to construct, understand, and modify?(造成程序代码难以构建, 理解, 以及修改的主要原因是什么?)

What is Computer?

A computer usually refers to an electronic device used for computing.

Computer = Hardware + Software

1.2 Focus of Software

The **focus** of software is on **algorithms**.

Each algorithm has three aspects:

1. **Functionality: system software (e.g., compiler, OS) and application software (e.g., AI systems, banking systems, ticket reservation systems, web systems)**
2. **Development and maintenance**
3. **Measurement of qualities (e.g., efficiency, usability, maintainability)**

1.3 Software Engineering

Research view:

Software Engineering is a field of computer science that studies theories and technologies for developing and maintaining large-scale software systems.

Practice view:

Software Engineering is a process to apply well-established software engineering technologies to building and maintaining large-scale software systems.

1.4 Trustworthy (Dependable) Software

Trustworthy is a composite quality or property that encompasses reliability, availability, safety, confidentiality (security, privacy), integrity, and maintainability

Dependable Software is software with high Dependability that allows trust and reliance to be justifiably placed on the service it delivers.

Software Reliability

Software reliability is the probability that software will operate without encountering errors during a given period of time.

Question for discussion:

For a given software system, do you think its reliability is a fixed value or a variable one?

Software Availability

Software availability is the proportion of time a system is up and running versus the total time it is expected to operate.

$$\text{Availability} = \text{Uptime} / (\text{Uptime} + \text{Downtime})$$

Question for discussion:

What can cause low software availability?

Software Safety

Software Safety refers to the **ability of software to operate without causing unacceptable risk or harm** to people, property, or the environment when it is used to control systems (e.g., airplanes, trains, automobiles).

Question for discussion:

Can software failure cause unacceptable risk or harm to people, property, or the environment if it is not connected to hardware systems?

Software Confidentiality

Software Confidentiality refers to the property of a software system that **ensures sensitive or private information is not disclosed to unauthorized users, systems, or processes** (e.g., bank accounts, private files).

Software Integrity

Software Integrity is the property of a software system that ensures its **data, code, and operations remain accurate, consistent, and unaltered by unauthorized or accidental changes.**

Question for discussion:

Is the software confidentiality the same as software integrity? If not, what is their relationship?

Software Maintainability

Software Maintainability is the quality attribute of a software system that measures **how easy it is to modify, update, and improve the software after its initial development and deployment.**

Question for discussion:

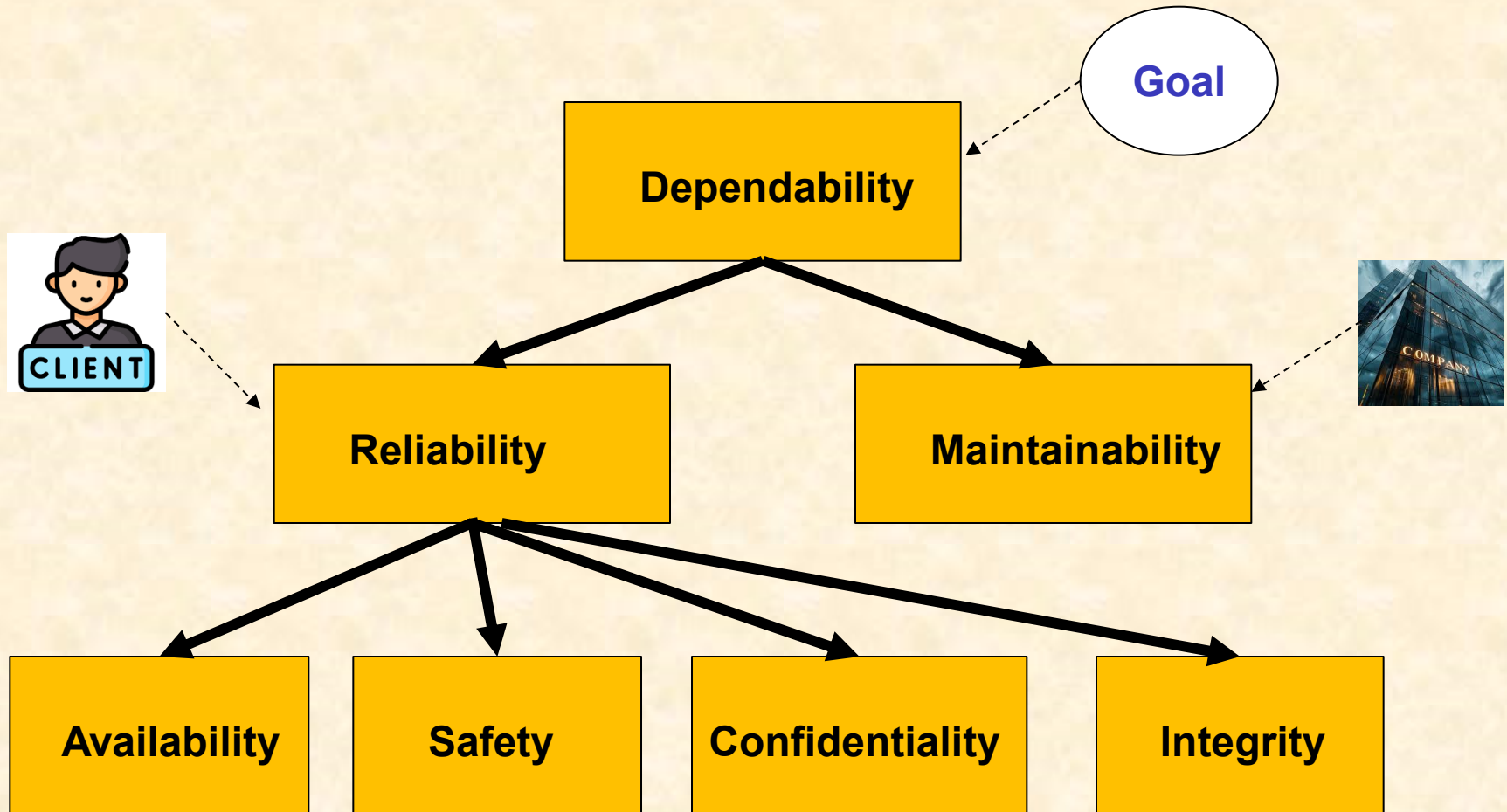
How do you usually determine if a software system has a low or high maintainability?

1.5 Trustworthy Software Engineering

Trustworthy Software Engineering studies theories and technologies for developing and maintaining large-scale software systems with high trustworthiness (dependability).

Goal (research): create and develop effective methods, languages, and supporting tools for producing software systems with high trustworthiness.

Important Goal in Trustworthy Software Engineering (practice)



Ensure:

Reliable =>

Safe, Available, Confidential, Integrous

Not Reliable => Safe, Confidential, Integrous

2. Technologies for Trustworthy Software Engineering

2.1 Challenges in Software Engineering

2.2 Existing Development Technologies

2.3 Formal Engineering Methods

2.4 Agile-SOFL: Agile Formal Engineering
Methods

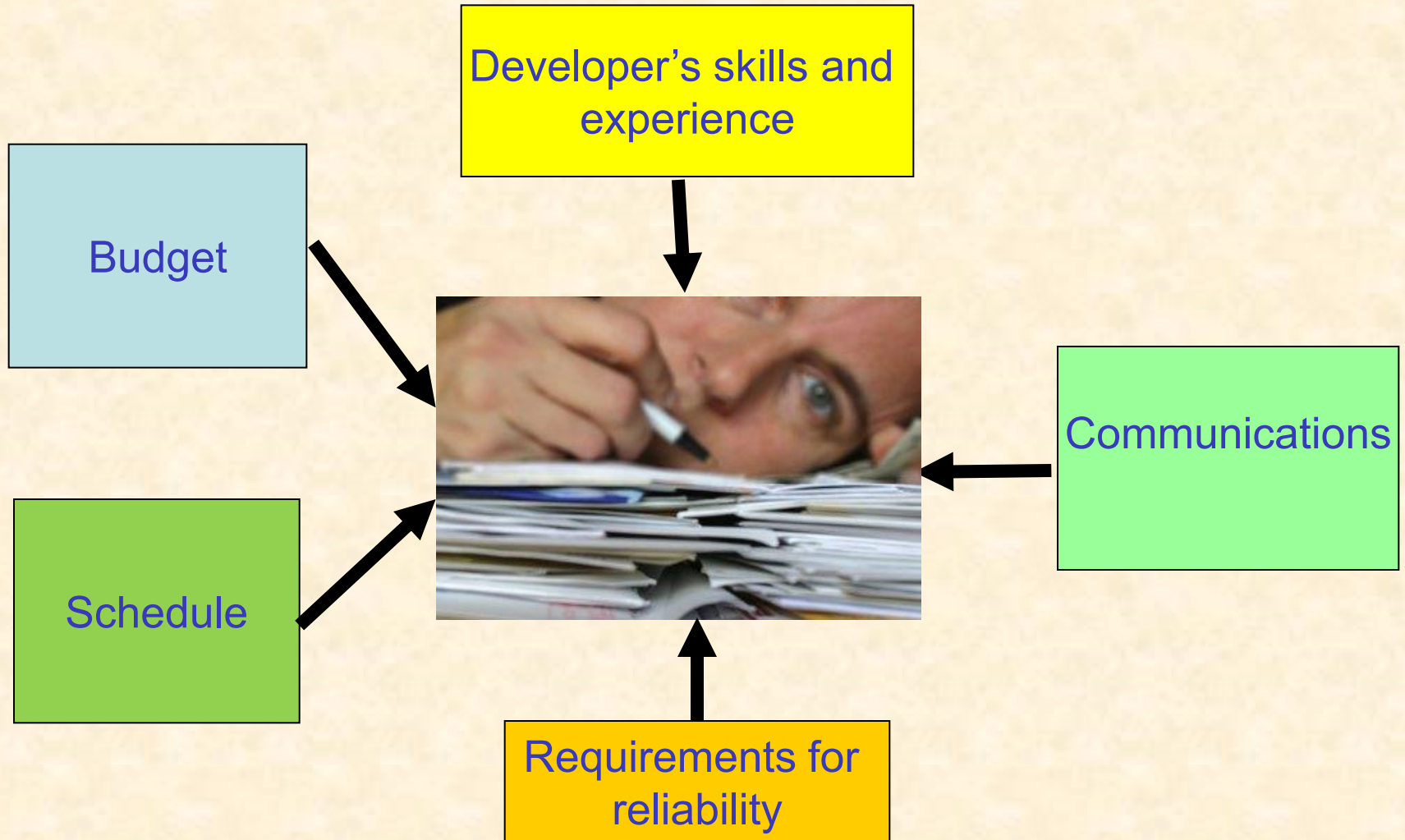
2.5 Testing-Based Formal Verification

2.6 Human-Machine Pair Programming

2.7 LLM-Based Software Engineering

2.1 Challenges in Software Engineering

Developers are usually required to achieve expected software products under many constraints.



The Reality of Software Development



Client:

Why does the software system behave differently from my requirements?



Manager:

Why is the project over budget and behind schedule?

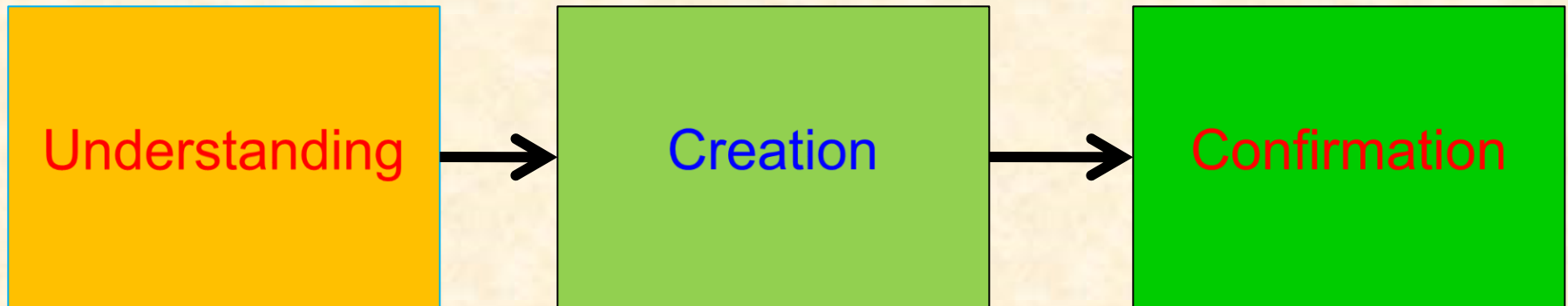


Programmer:

Why are there so many bugs remaining in the program?

Why is my own program difficult to understand even by myself?

Three major activities in software engineering:



2.2 Existing Development Technologies

- ❑ Model-Driven Development (MDD)
- ❑ Formal Methods (FM)
- ❑ Component-Based Software Engineering (CBSE)
- ❑ Agile Development (AD)

(1) Model-Driven Development (MDD)

Characteristics:

- Building models before coding (e.g., UML)
- Documentation but not formal
- Understanding before implementation
- No well-defined software development process

Problems:

- Time consuming for building models, documentation evolution, and configuration management (for consistency between versions)

(2) Formal Methods (FM)

Formal methods mean **verified design**.

Formal methods =

Formal Specification (VDM, Z, B-Method)

+

Refinement (Back, Morgan)

+

Formal Verification (Floyd-Hoare, Dijkstra)



Set theory, logics, algebra, etc.

Characteristics of Formal Methods:

- Precise documentation for good understanding of the system (at least by the analyst himself)
- There is a well-defined refinement process for software development
- Solid foundation for transformation and verification, as well as for automation

Problems:

- Formal methods face the same problems as model-driven approaches.
- Complexity in mathematical manipulation
- Capability limitations in dealing with software engineering issues (validation of specifications, communications, side effect, culture, management etc.)

(3) Component-Based Software Engineering (CBSE)

1. Core Idea:

- ◆ Software systems are built by **integrating software components**, reducing development time and improving maintainability.
- ◆ A **component** is a self-contained, modular piece of software with a well-defined interface (e.g., classes and packages in java.util).
- ◆ Components can be **reused** across different applications.

2. Key Characteristics of Components

- **Encapsulation:** Internal details are hidden; only the interface is visible.
- **Reusability:** Can be reused in multiple systems without modification.
- **Replaceability:** Components can be swapped without affecting other parts of the system.
- **Composability:** Components can be combined to form larger systems.

3. Benefits

- Faster development due to reuse of existing components.
- Higher reliability because tested components are reused.
- Easier maintenance: replace or upgrade components without rewriting the system.
- Supports modular architecture and scalability.

4. Challenges

- Finding suitable components that match requirements.
- Integrating components from different vendors or with different standards.
- Managing dependencies between components.
- Performance overhead due to abstraction and communication between components.

(4) Agile Development (AD)

Background:

Characteristics of software engineering:

Large-scale (software development)

Evolutionary (during software development)

human-driven (in software development)

error-prone (during software development)

Agile manifesto

- (1) Individuals and interactions over processes and tools**
- (2) Working software over comprehensive documentation**
- (3) Customer collaboration over contract negotiation**
- (4) Responding to change over following a plan**

Characteristics of AD:

- Evolutionary-prototyping based
- User-centered
- Test-driven
- eXtreme Programming (XP)
- No documentation except code
- Quick releases

Problems:

- Frequent changes of code
- Difficulty in understanding code

2.3 Formal Engineering Methods

-Addressing Practitioners' dilemma-



Happy
world



Formal
Methods



Software
practitioner



Traditional SE

What are Formal Engineering Methods (FEM)

Formal Engineering Methods (FEM) provide ways to integrate Formal Methods and conventional software engineering techniques to achieve rigor (methodology), comprehensibility (for humans), and tool supportability (tool) for those techniques.

**Formal
Methods**



**Software
Engineering**

**Formal
Engineering
Methods**

The SOFL method – a Specific FEM

SOFL stands for Structured Object-Oriented
Formal Language

SOFL = Language + Method

The SOFL specification language

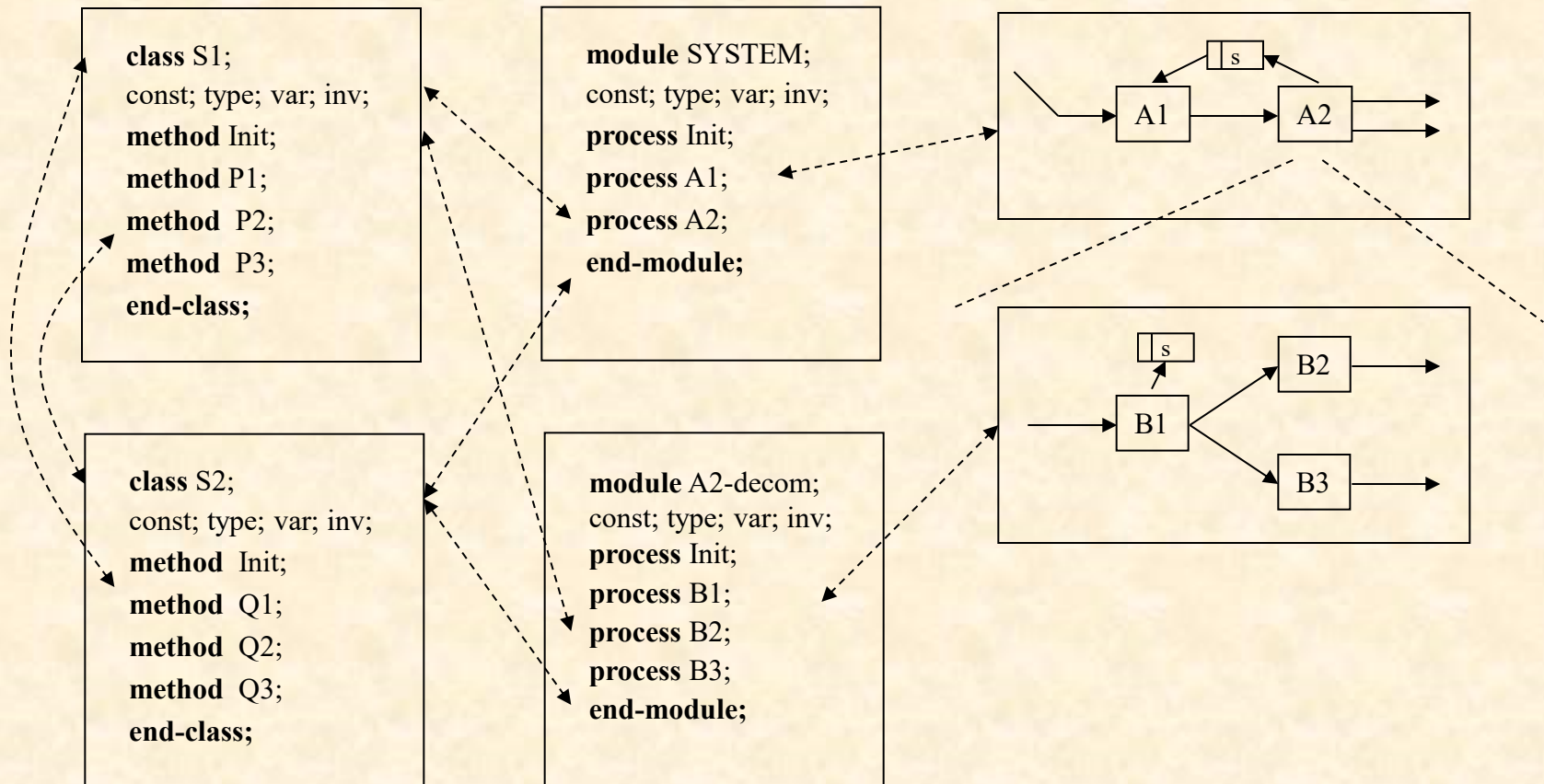
As a language, SOFL is an integration of the following three notations:

- (1) VDM-SL (VDM Specification Language)
- (2) Petri Nets
- (3) Data Flow Diagrams (DFD)

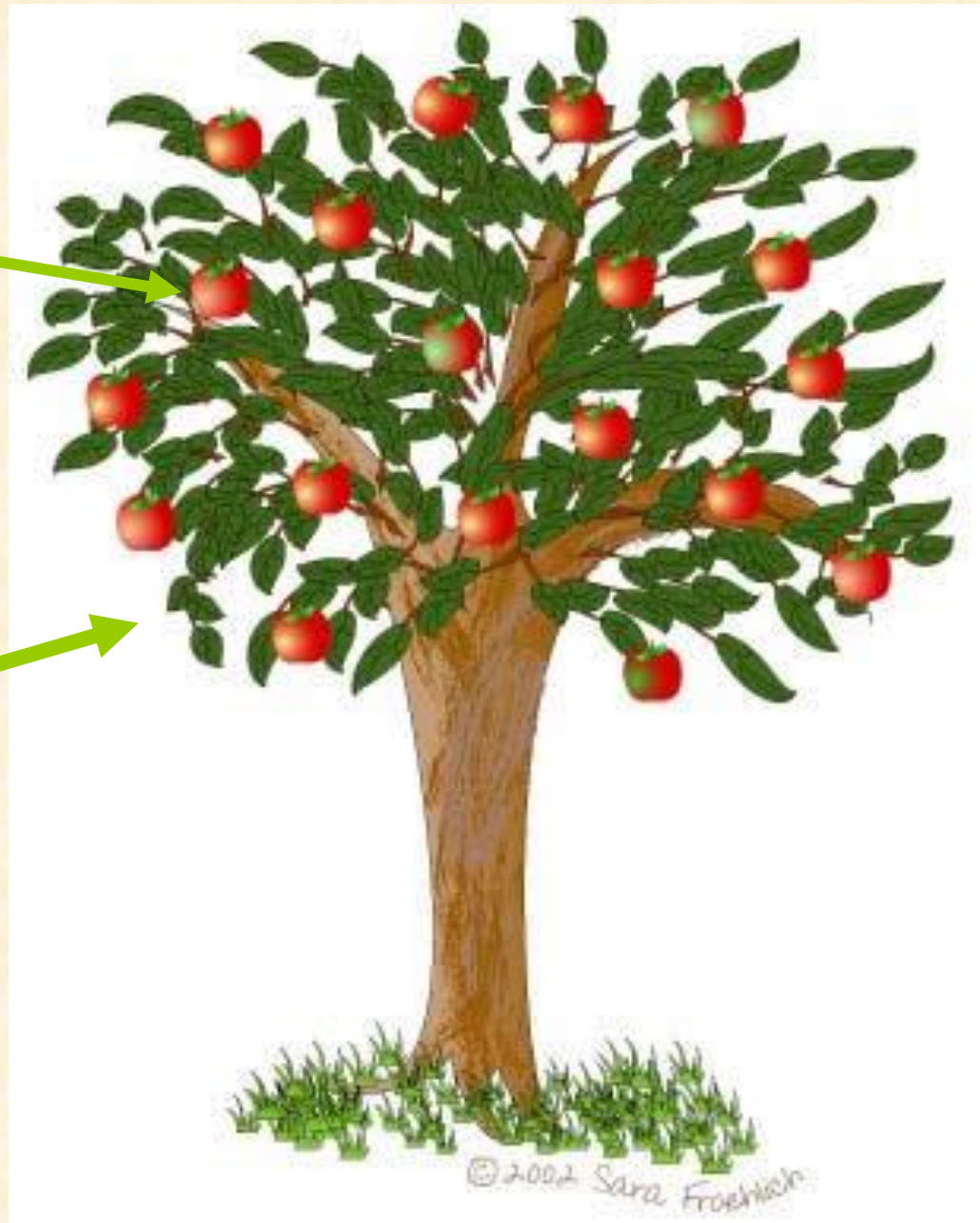
The DFD provides a comprehensible architecture for specifications; Petri Nets are utilized to provide an operational semantics for the DFD; and VDM-SL is used for defining the components of the formalized DFD, such as processes, data flows, and data stores.

The structure of a SOFL specification:

CDFDs + modules + classes



Component



Architecture

The SOFL Method

The SOFL three-step formal
specification technique

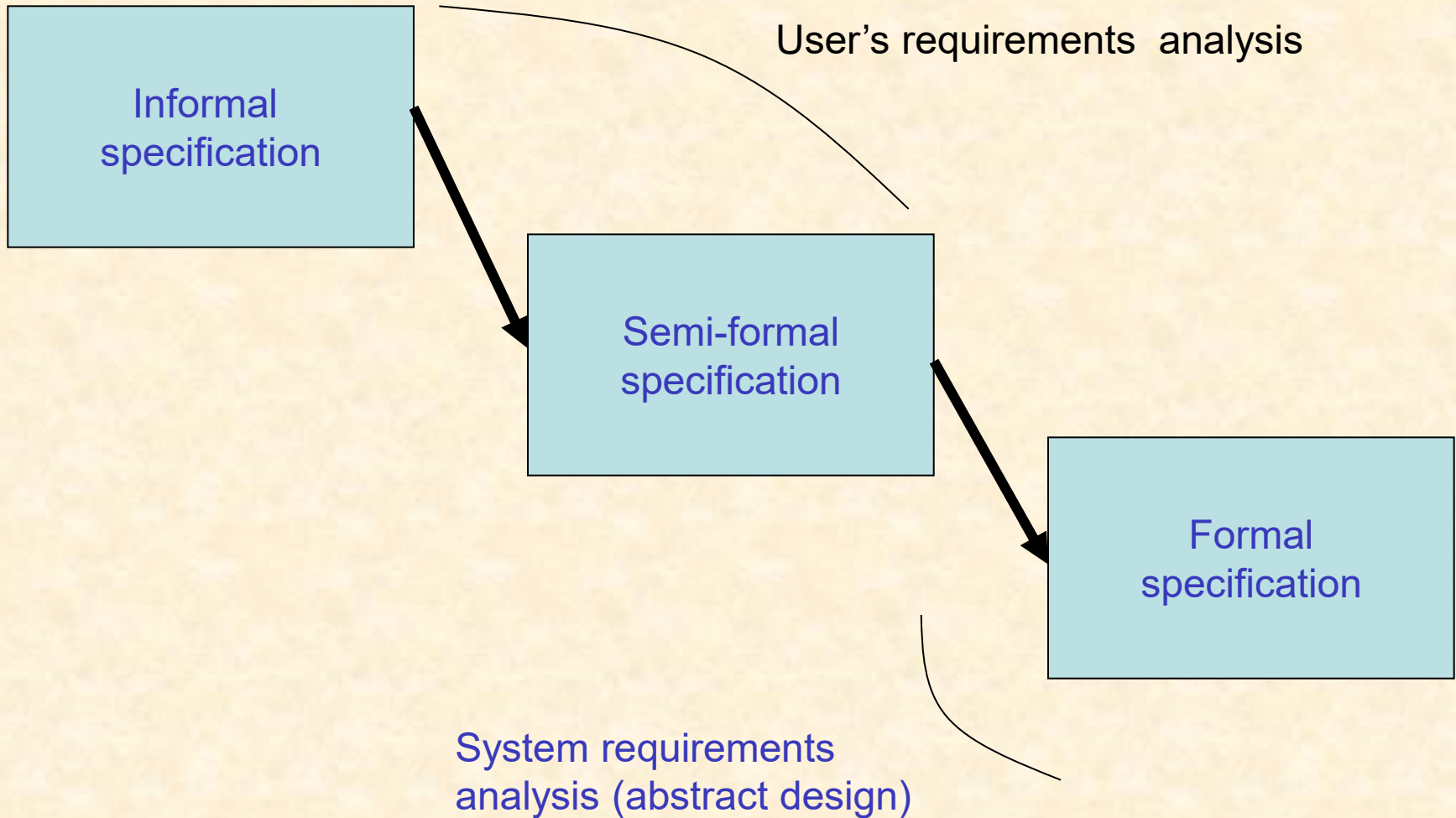


**Software
defects and
errors**

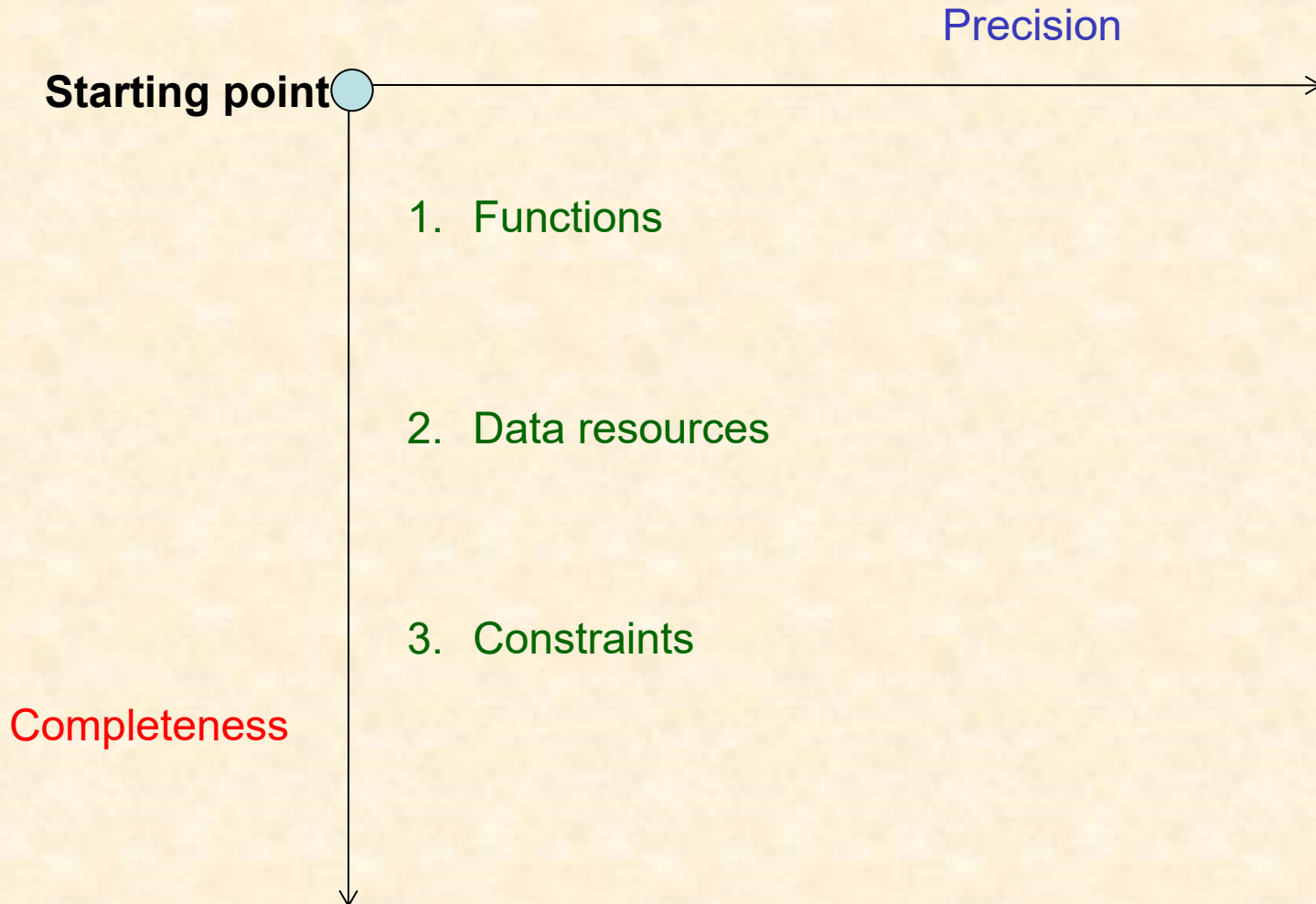
Specification-
based Program
Inspection

Specification-
based Program
Testing

(1) SOFL Three-Step Formal Specification Technique



Tasks for informal specification: **Capturing** desired **functions**, necessary **data resources**, and **constraints** on both functions and data resources.



Example: A simplified ATM system description

The ATM system **responds** to the **user's requests** properly by providing correct services. The services include **withdrawing** money from the user's account and **displaying** the account balance. When any of such a service is requested, the user's **id and password** are **checked** to ensure that the user will **not access** other **customer's account**. It is the policy of the bank that requires that **no customer (user) can borrow money from the bank**.

[1] Informal specification (User's requirements elicitation):

(1) Functions:

- Receive request from the customer
- Check the customer's card id and password
- Withdraw
- Account balance inquiry

(2) Data resources:

- Customers' account information

(3) Constraints:

- A customer cannot access other person's account.
- No customer is allowed to borrow money from the bank.

Hierarchy of informal specification:

1. The functions of the system:

(1) F1

(2) F2

(3) F3

(4) F4

1.1 F1

(1) F11

(2) F12

(3) F13

1.2 F2

(1) F21

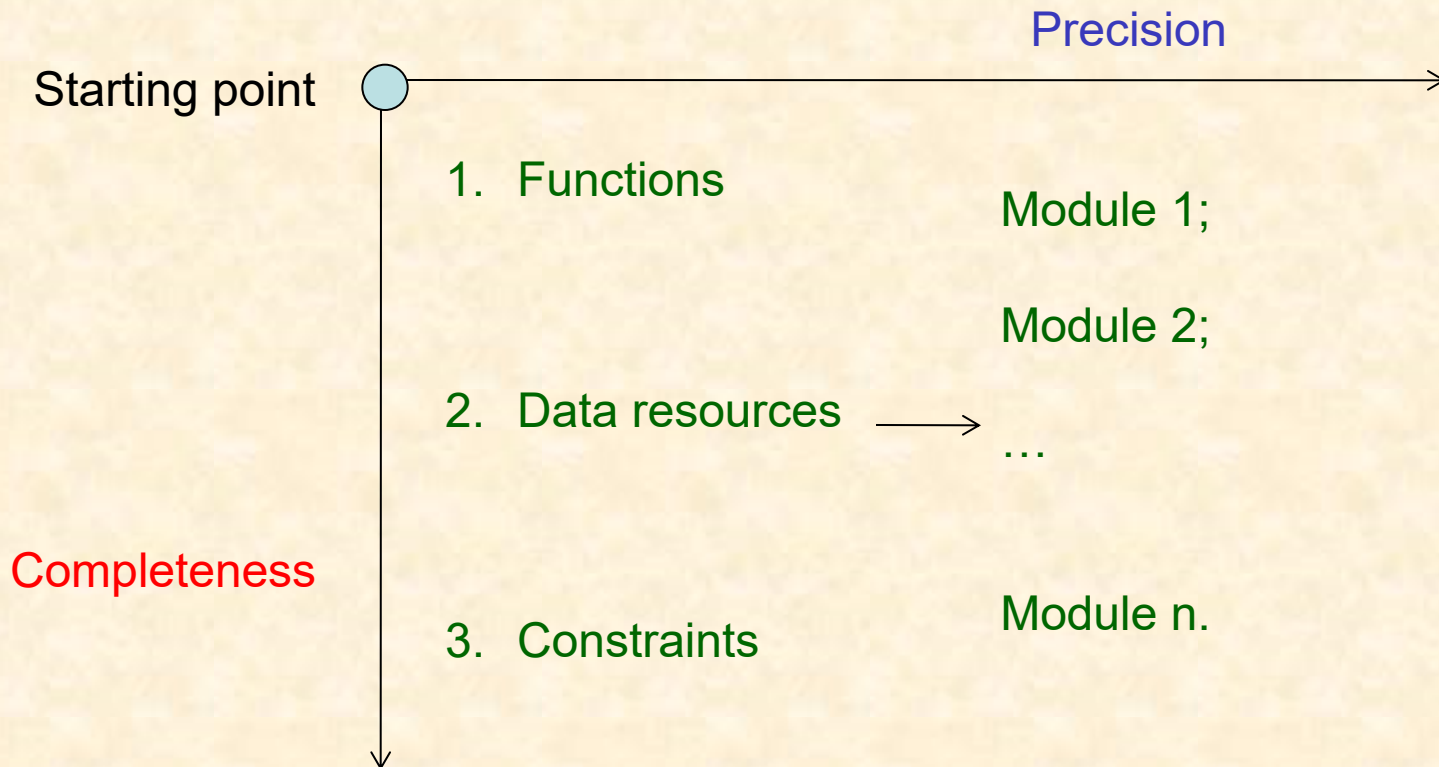
(2) F22

● ● ● ● ● ●

2. Recourses:

3. Constraints:

Tasks for semi-formal specification: (1) **Grouping** related functions, data resources, and constraints into SOFL modules. (2) **Defining** necessary data types and variables. (3) **Defining** the function of each process (operation) using pre- and post-conditions at informal level.



Example:

[2] Semi-formal specification (User's requirements analysis and description):

```
module SYSTEM_ATM
```

```
  type
```

```
    Account = composed of
```

```
        account_no: nat
```

```
        password: nat
```

```
        balance: real
```

```
    end
```

```
  var
```

```
    account_file: set of Account;
```

```
  inv
```

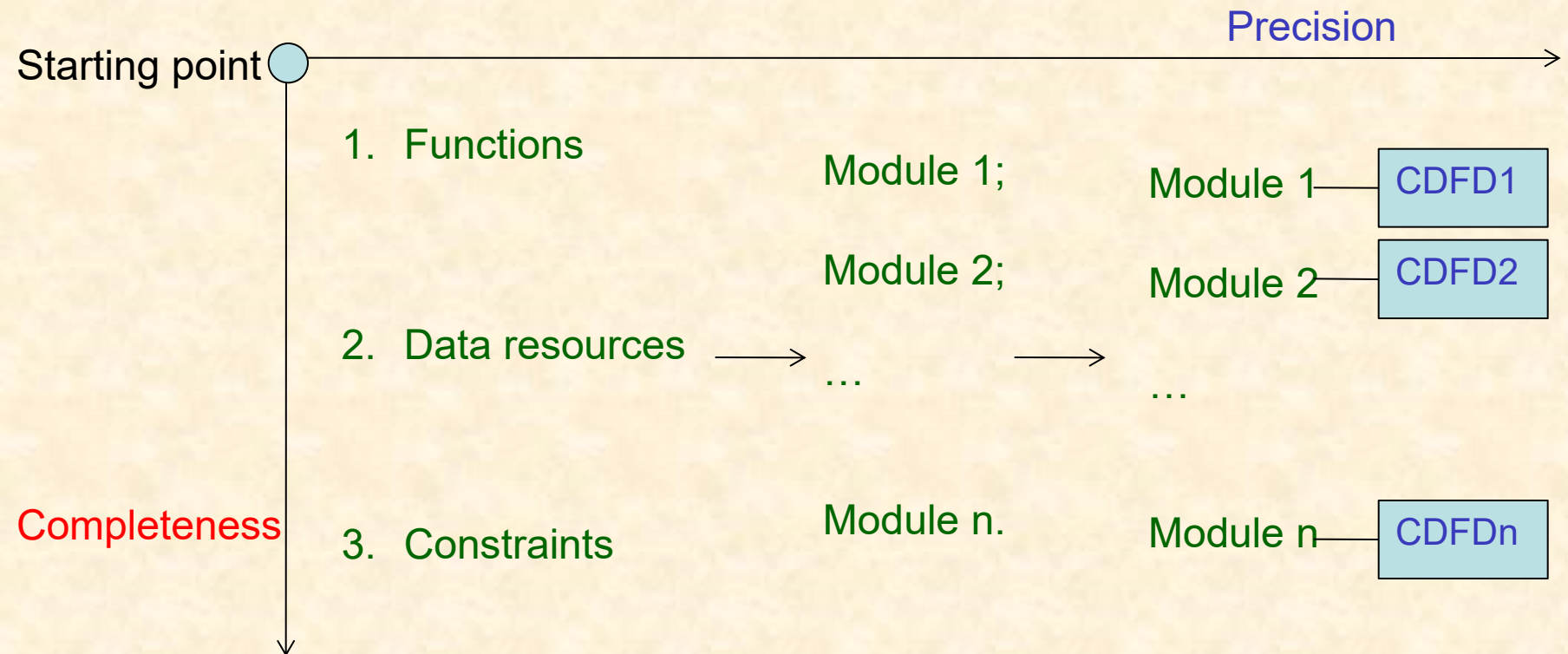
Account balance must be greater than or equal to zero.

```
process Withdraw(amount: real, account1: Account)
    e_msg: string | cash: real
ext wr account_file
pre account1 is a member of account_file
post if amount is less than the balance of account1
    then supply cash with the same amount as amount, and
        reduce the amount from the balance of the account.
    else output an appropriate error message e_msg.
end_process;

...

end_module
```

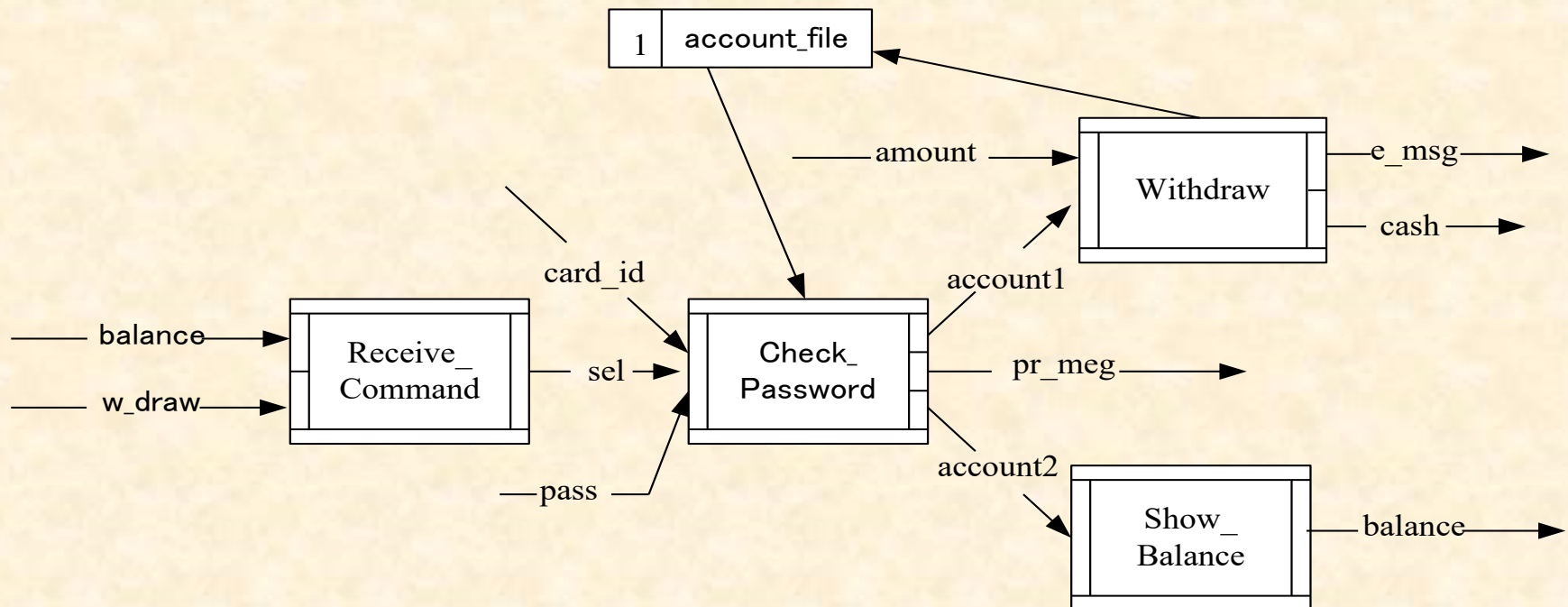
Tasks for formal specification: (1) **Describing system architecture** using hierarchical CDFDs (condition data flow diagram). (2) **Formalizing** the specification of each operation in pre- and post-conditions.



Example:

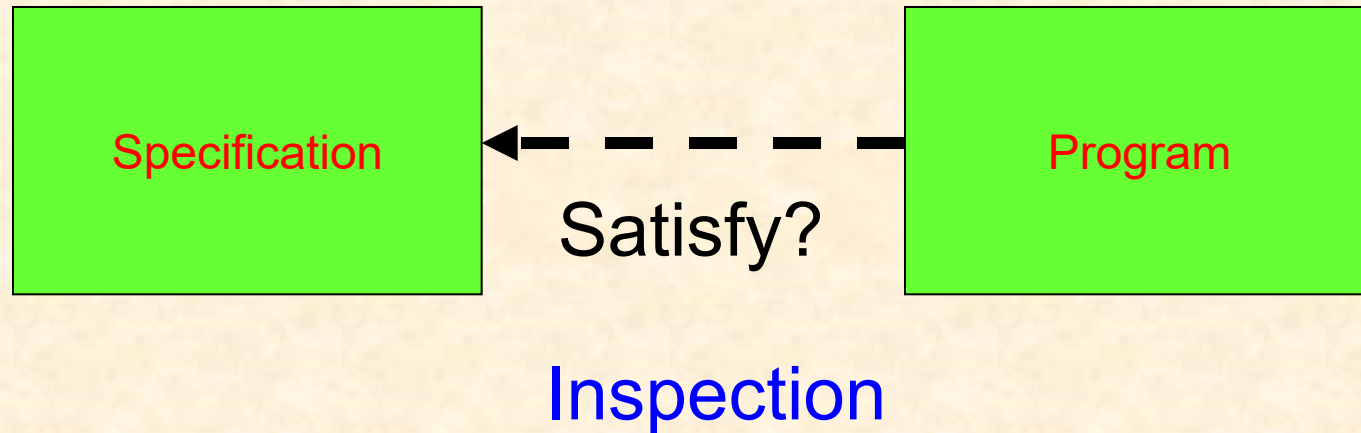
[3] Formal specification (System requirements analysis and definition):

The top level CDFD:



```
process Withdraw(amount: real, account1: Account)
    e_msg: string | cash: real
ext wr account_file
pre account1 inset account_file
post if amount <= account1.balance
    then
        cash = amount and
        let Newacc =
            modify(account1, balance -> account1.balance – amount)
        in
            account_file = union(diff(~account_file, {account1}), {Newacc})
    else
        e_meg = "The amount is over the limit. Reenter your amount."
comment
...
end_process;
end_module
```


(2) Specification-Based Inspection



Why inspection before testing?

(1) Testing activities are usually time-consuming and costly.

(2) Testing may not be efficiently performed (crash, non-termination etc.)

Scenario-based inspection: a strategy for “divide and conquer”

Specification

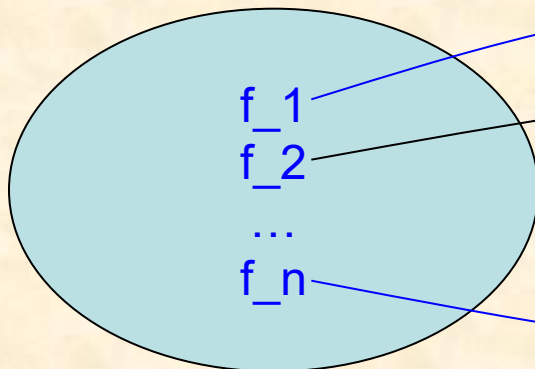
```
process A(x: int) y: int
pre  x > 0
post x > 10 and y = x + 1 or
     x <= 10 and y = x - 1
end_process
```

Functional scenario:

$A_{pre} \wedge G_i \wedge D_i$

$(i=1, \dots, n)$

Functional scenarios

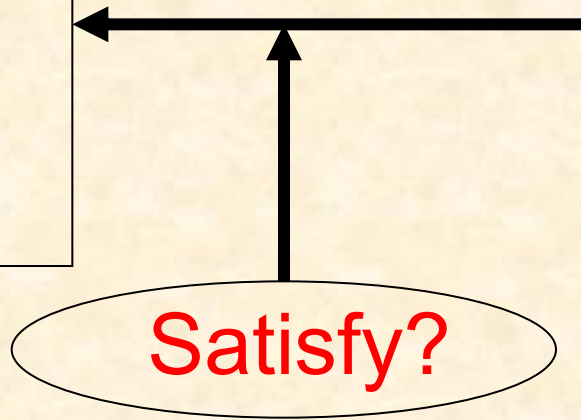
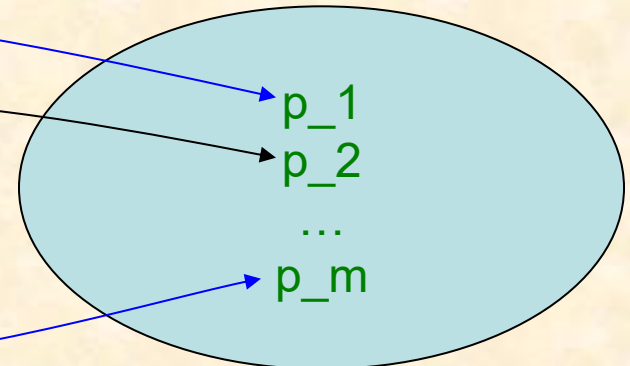


M_i

Program

```
int A(int x) {
  If (x > 0) {
    if (x > 10) y = x + 1;
    else y = x - 1;
    return y; }
  else System.out.println("the
    pre is violated") }
```

Program paths



...

The principle of scenario-based inspection:

- (1) Check whether every functional scenario defined in the specification is correctly implemented by a set of program paths in the program.
- (2) Every program path contributes to the implementation of some scenario.

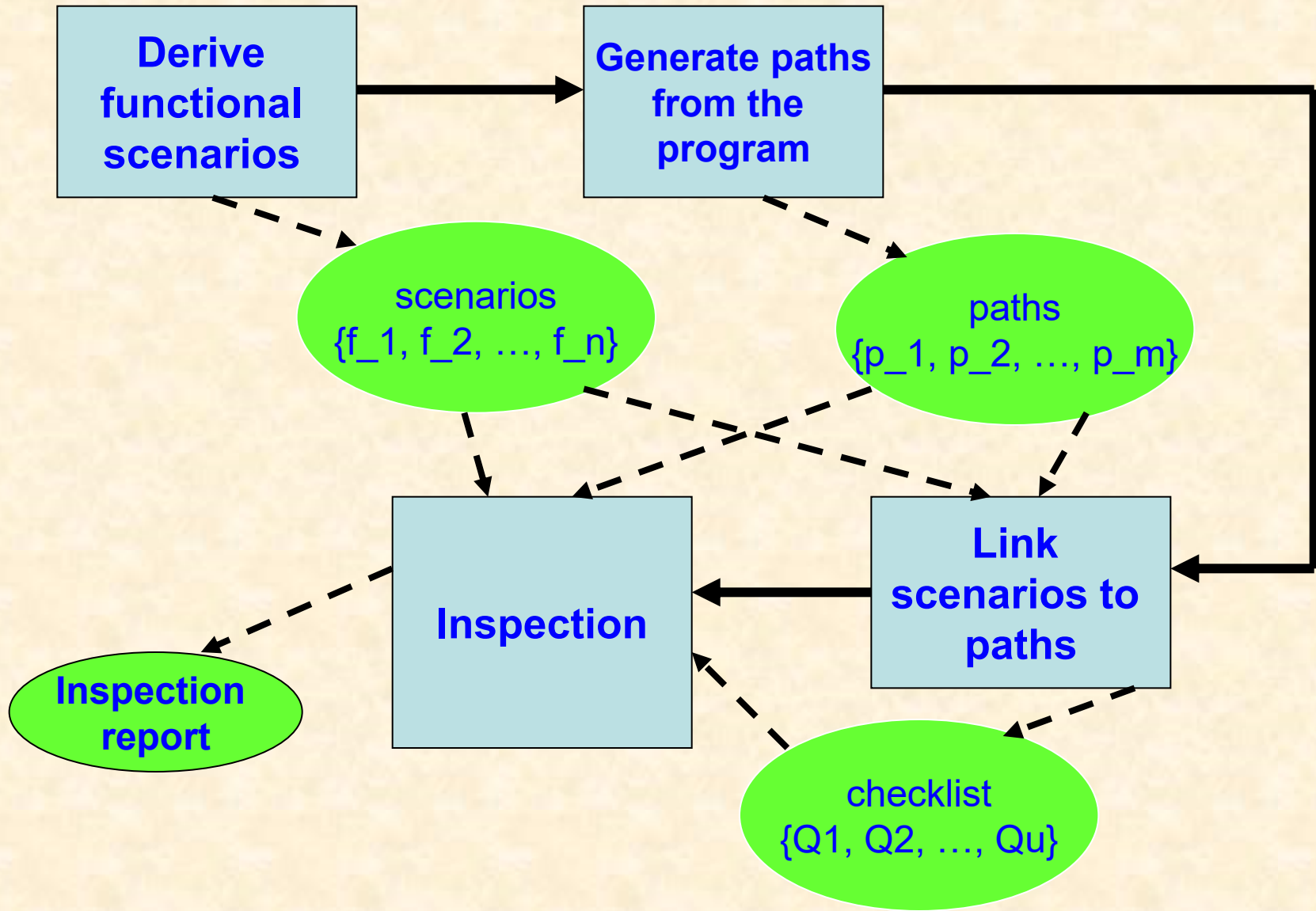
Formally, the principle is described as follows:

$M: S \rightarrow \text{power}(P)$

$(\forall f \in S \exists q \in \text{power}(P) \cdot M(f) = q) \wedge$

$(\forall p \in P \exists f \in S \cdot p \in M(f))$

A Process for Scenario-Based Inspection



Inspection target list

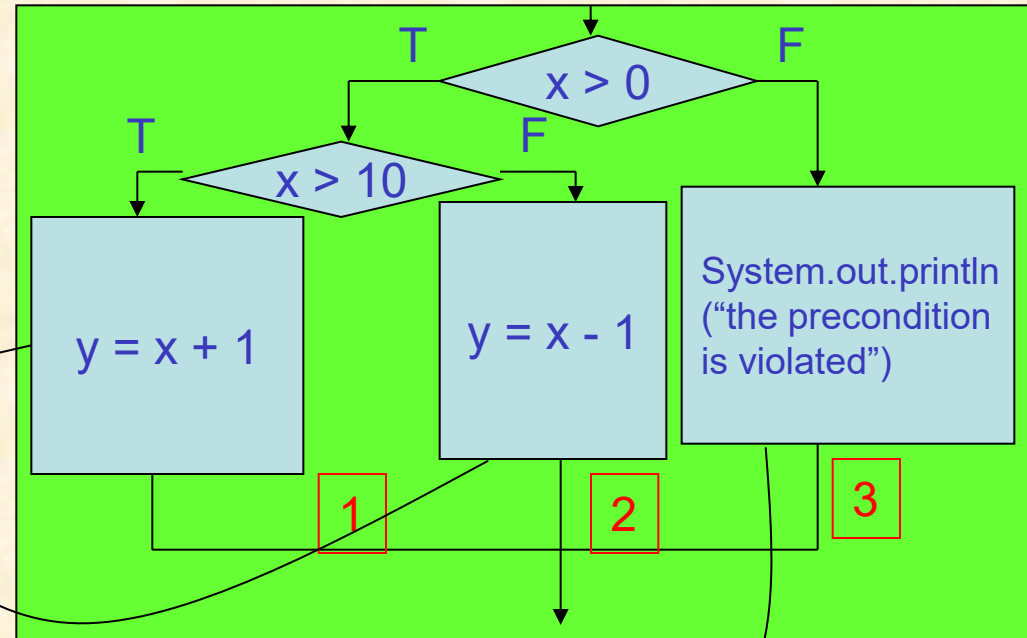
Specification

A(x: int) y: int
pre $x > 0$
post $x > 10 \wedge y = x + 1 \vee$
 $x \leq 10 \wedge y = x - 1$

Functional scenarios:

- (1) $x > 0 \wedge x > 10 \wedge y = x + 1$
- (2) $x > 0 \wedge x \leq 10 \wedge y = x - 1$
- (3) $x > 0$ (optional)

Program



Inspection

Checklist (specific for process A):

- Is the precondition $x > 0$ of the specification properly implemented in the program?
- Is the guard condition $x > 10$ implemented properly in the program?
- Is the guard condition $x \leq 10$ implemented properly in the program?
- Is the defining condition $y = x + 1$ implemented properly in the program?
- Is the defining condition $y = x - 1$ implemented properly in the program?
- ...

(3) Specification-Based Testing

The goal:

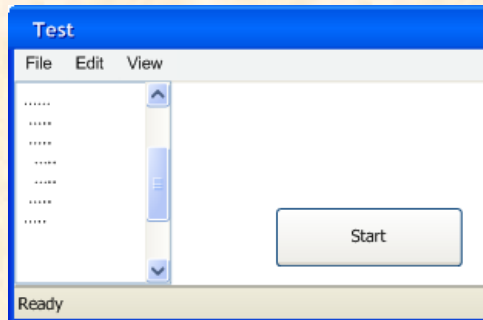
Dynamically check whether the functions defined in the specification are ``correctly'' implemented by the program.

A program **P** correctly implements a specification **S** iff

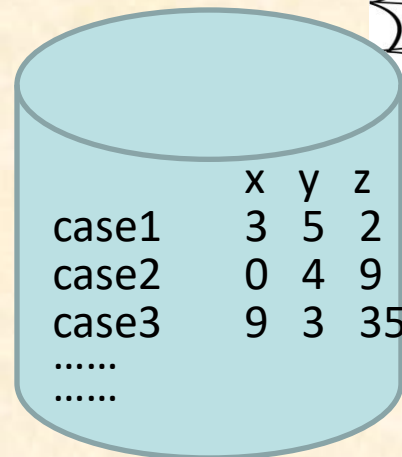
$$\forall \sim\sigma \in \Sigma \cdot S_{\text{pre}}(\sim\sigma) \Rightarrow S_{\text{post}}(\sim\sigma, P(\sim\sigma))$$

The Goal of Automatic Specification-Based Testing

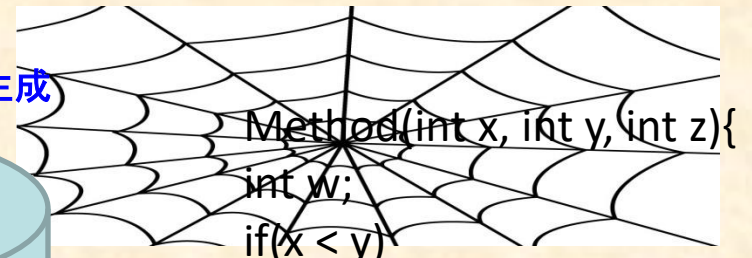
Press a Button



テストケースの自動生成



	x	y	z
case1	3	5	2
case2	0	4	9
case3	9	3	35
.....			
.....			



```
Method(int x, int y, int z){  
    int w;  
    if(x < y)  
    {  
        w = y/x;  
        while(w < z)  
        {  
            ...  
        }  
    } else  
    {  
        ...  
    }  
}
```



Next

Steps of specification-based testing:

- (1) Generate test cases from the specification.
- (2) Execute the program using the test cases.
- (3) Analyze test results to detect bugs based on the test case, the result of execution, and the specification.

General criteria for test case generation and for test result analysis:

Definition 5.1 (FSF)

Let $S_{\text{post}} \equiv G_1 \wedge D_1 \vee G_2 \wedge D_2 \vee \dots \vee G_n \wedge D_n$,

G_i : guard condition

D_i : defining condition.

$i = 1, \dots, n$.

Then, a functional scenario form (FSF) of S is:

$$(S_{\text{pre}} \wedge G_1 \wedge D_1) \vee (S_{\text{pre}} \wedge G_2 \wedge D_2) \vee \dots \vee (S_{\text{pre}} \wedge G_n \wedge D_n)$$

Criterion 5.1: Let an FSF of specification **S** be:

$$(S_{pre} \wedge G_1 \wedge D_1) \vee (S_{pre} \wedge G_2 \wedge D_2) \vee \dots \vee \\ (S_{pre} \wedge G_n \wedge D_n)$$

Then, a test set **T** must be generated to meet the following condition:

$$(\forall G_i \exists t \in T \cdot S_{pre} \wedge G_i(t)) \wedge \\ (\exists t \in T \cdot \neg S_{pre})$$

where $i = 1, \dots, n$

A criterion for test result analysis:

Criterion 5.2: If the condition

$$\exists t \in T \cdot S_{\text{pre}}(t) \wedge \neg S_{\text{post}}(t, P(t))$$

holds, it will indicate the existence of bugs in program **P**, where **T** is a test set generated from **S**.

Test case generation

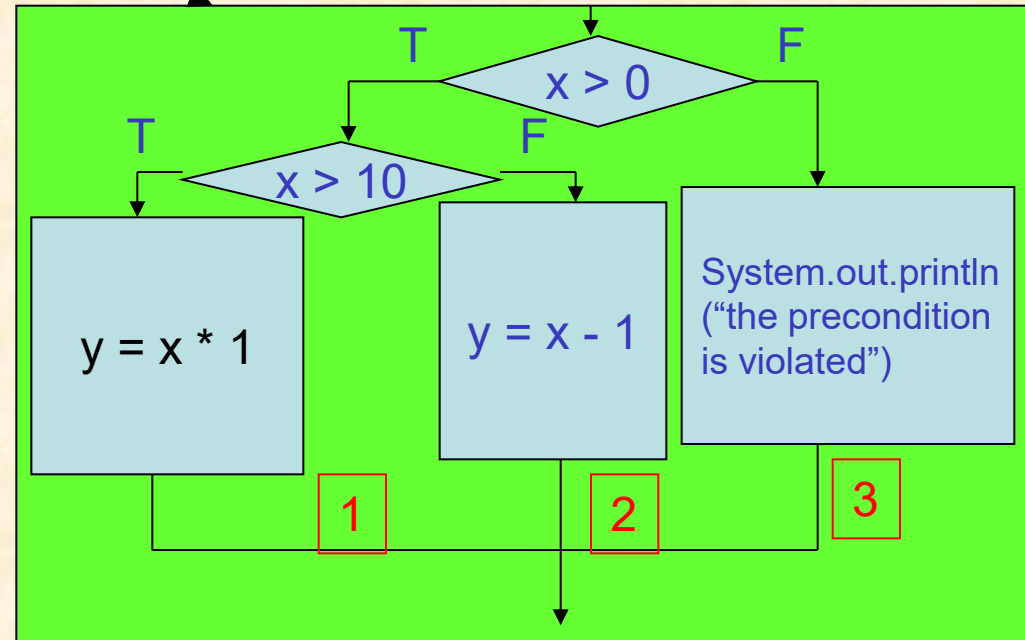
Specification

$A(x: \text{int}) y: \text{int}$
pre $x > 0$
post $x > 10 \wedge y = x + 1 \vee$
 $x \leq 10 \wedge y = x - 1$

Functional scenarios:

- (1) $x > 0 \wedge x > 10 \wedge y = x + 1$
- (2) $x > 0 \wedge x \leq 10 \wedge y = x - 1$
- (3) $x > 0$ (optional)

Program



Test result analysis

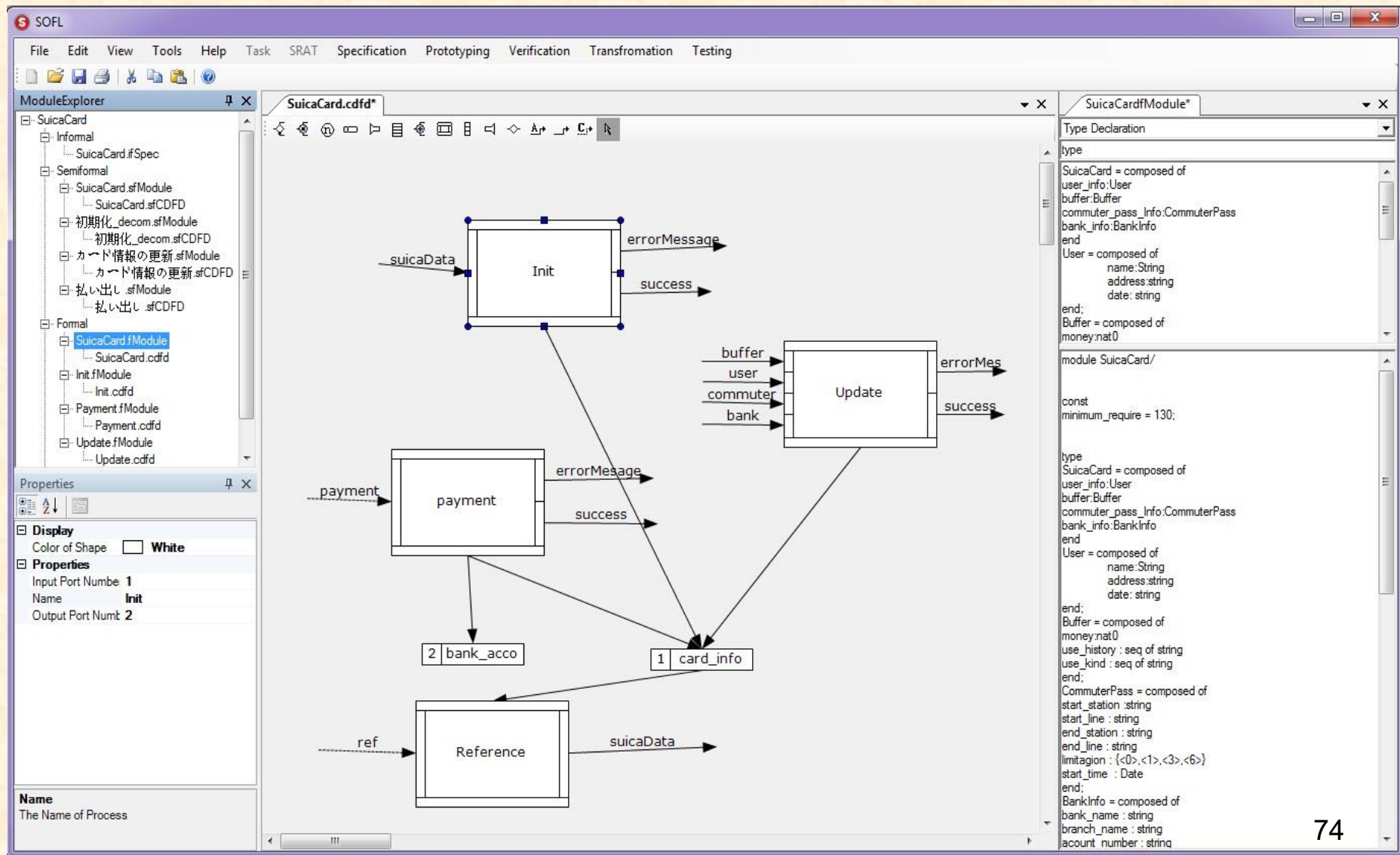
x	y	Ap _{pre}	Ap _{ost}	Ap _{pre} ∧ ¬ Ap _{ost}
15	15	true	false	true
5	4	true	true	false

Tool Support for SOFL

We have several prototype tools to support the SOFL specification language and method.

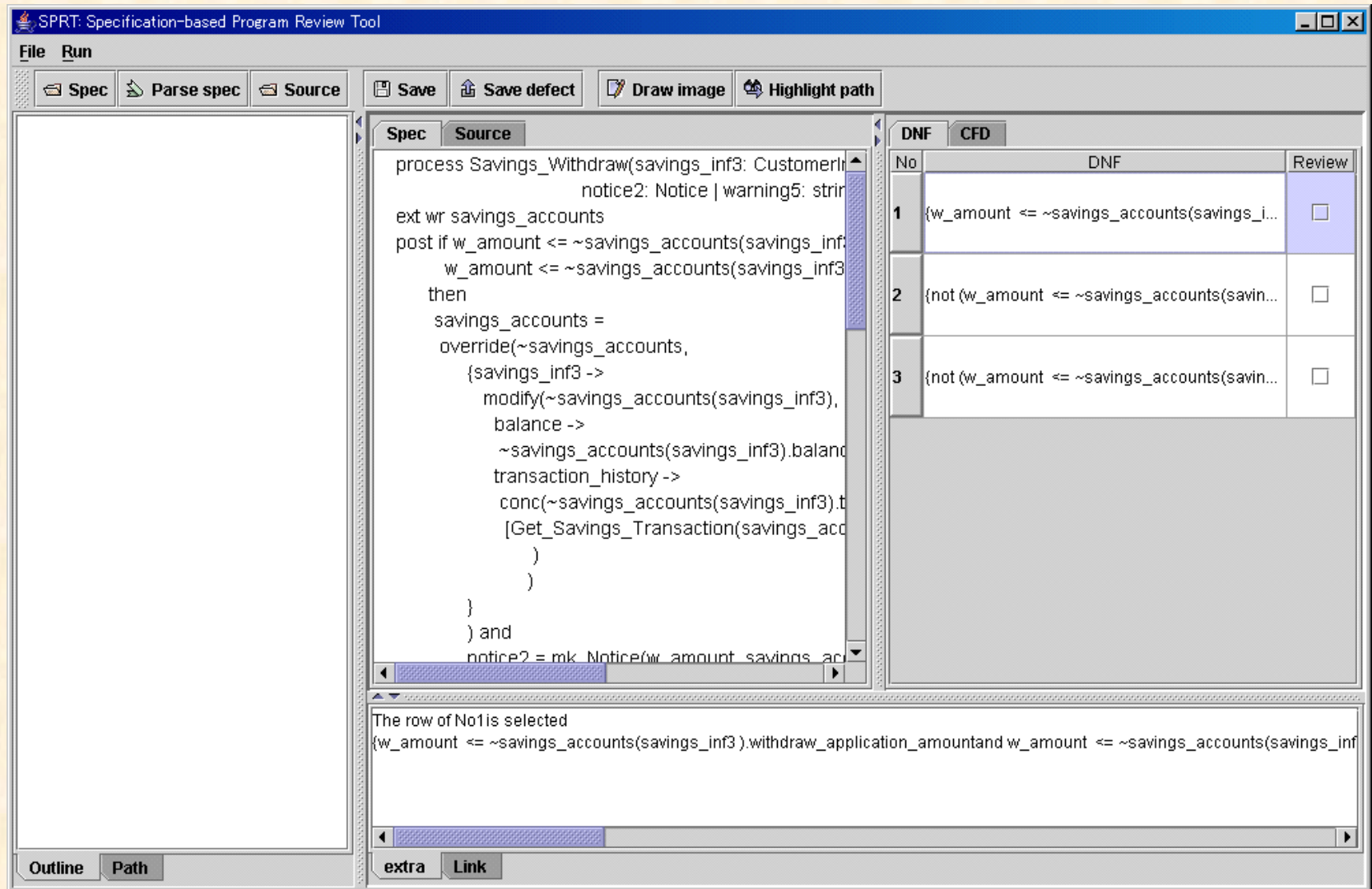
- SOFL specification construction tool (SpecTool)
- Specification-based program inspection tool (SBPIT)
- Specification-based testing tool (SBTT)

SpecTool for SOFL specification

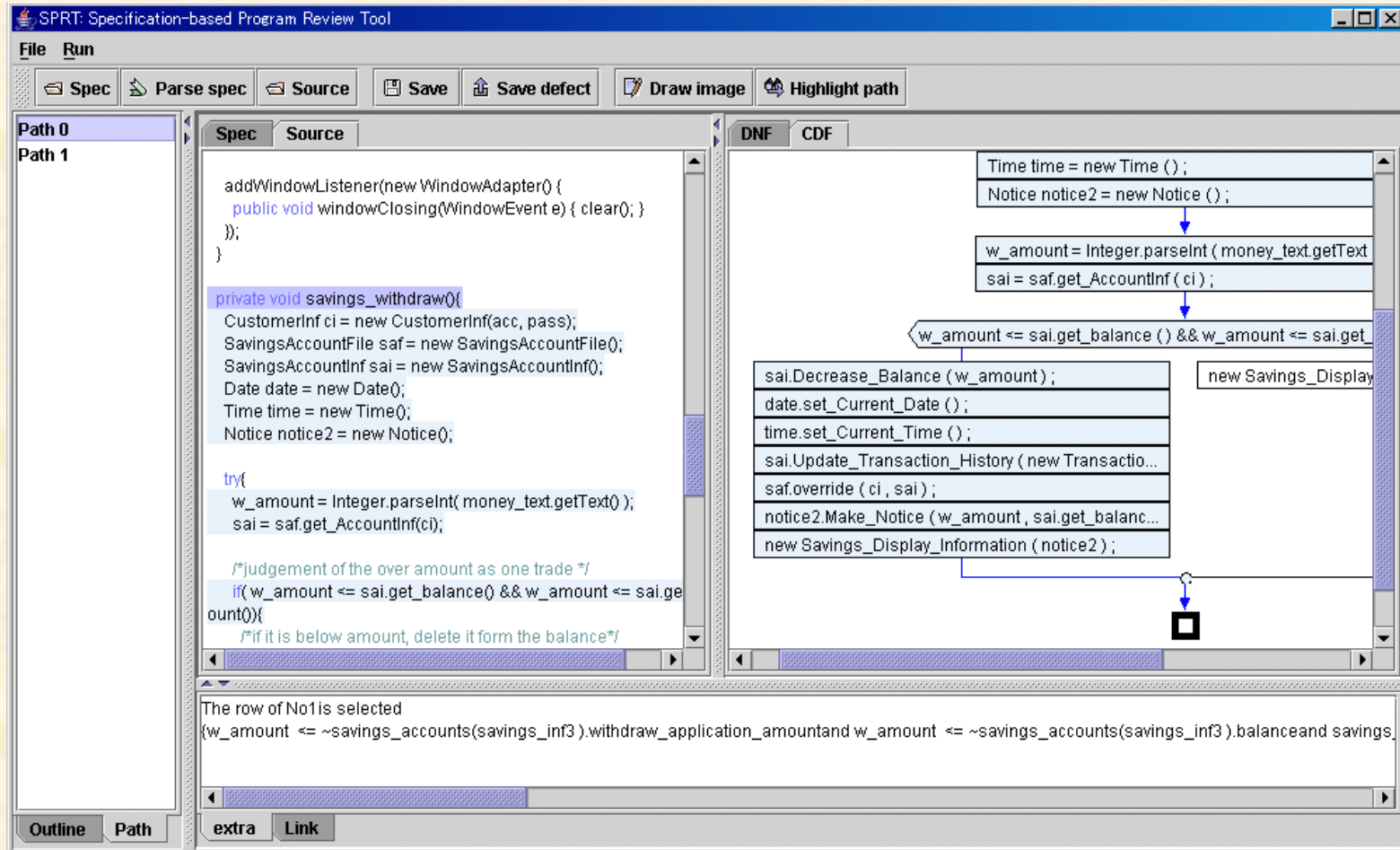


Specification-based program inspection tool (SBPIT)

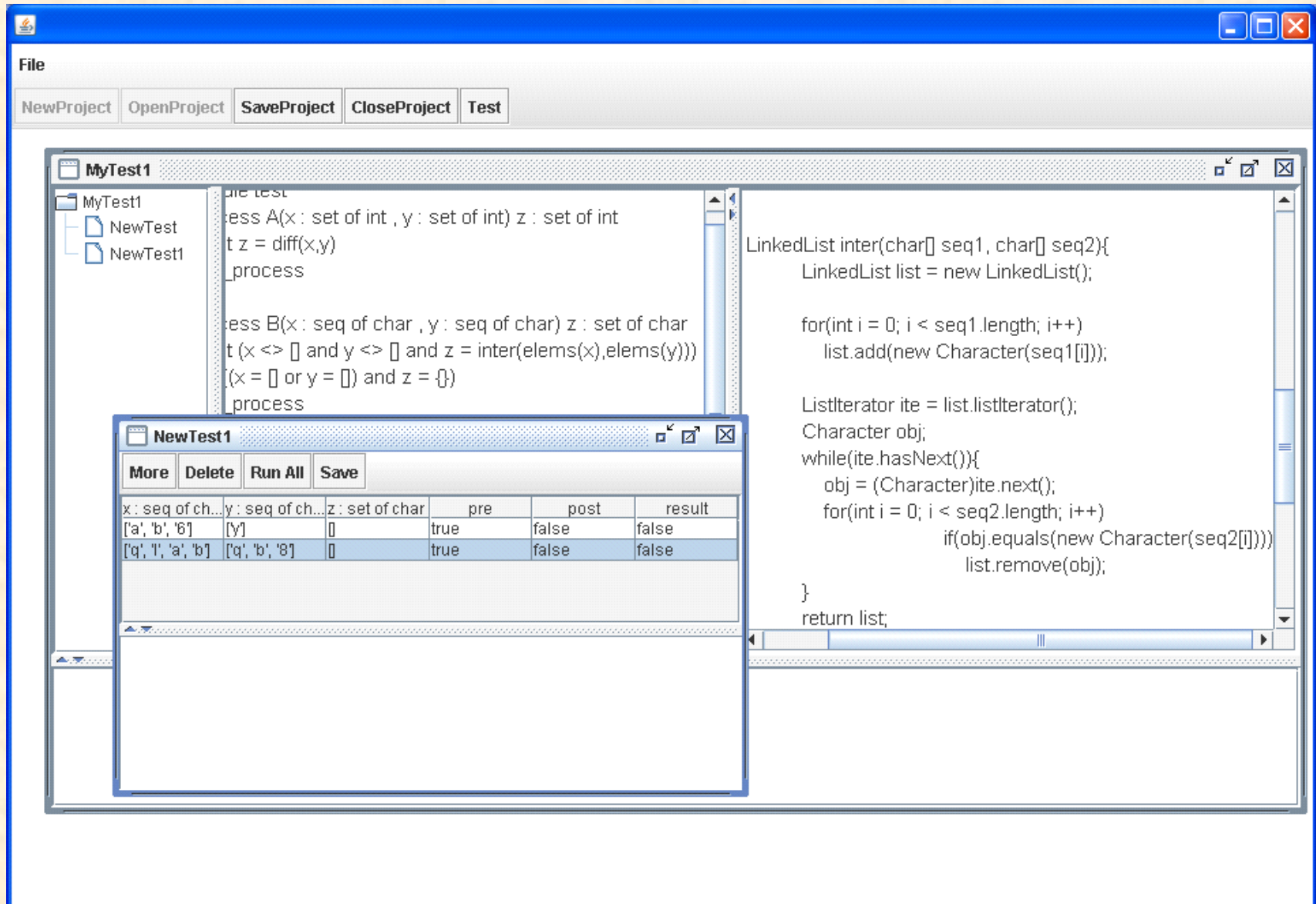
(Automatic generation of functional scenarios)



(Automatic generation of program paths)



Specification-based testing tool (SBTT)



SOFL Book

“**Formal Engineering for Industrial Software Development Using the SOFL Method**”,

by **Shaoying Liu**,
Springer-Verlag, 2004,
ISBN 3-540-20602-7

软件开发的形式化工程方法 –
结构化+面向对象+形式化
清华大学出版社

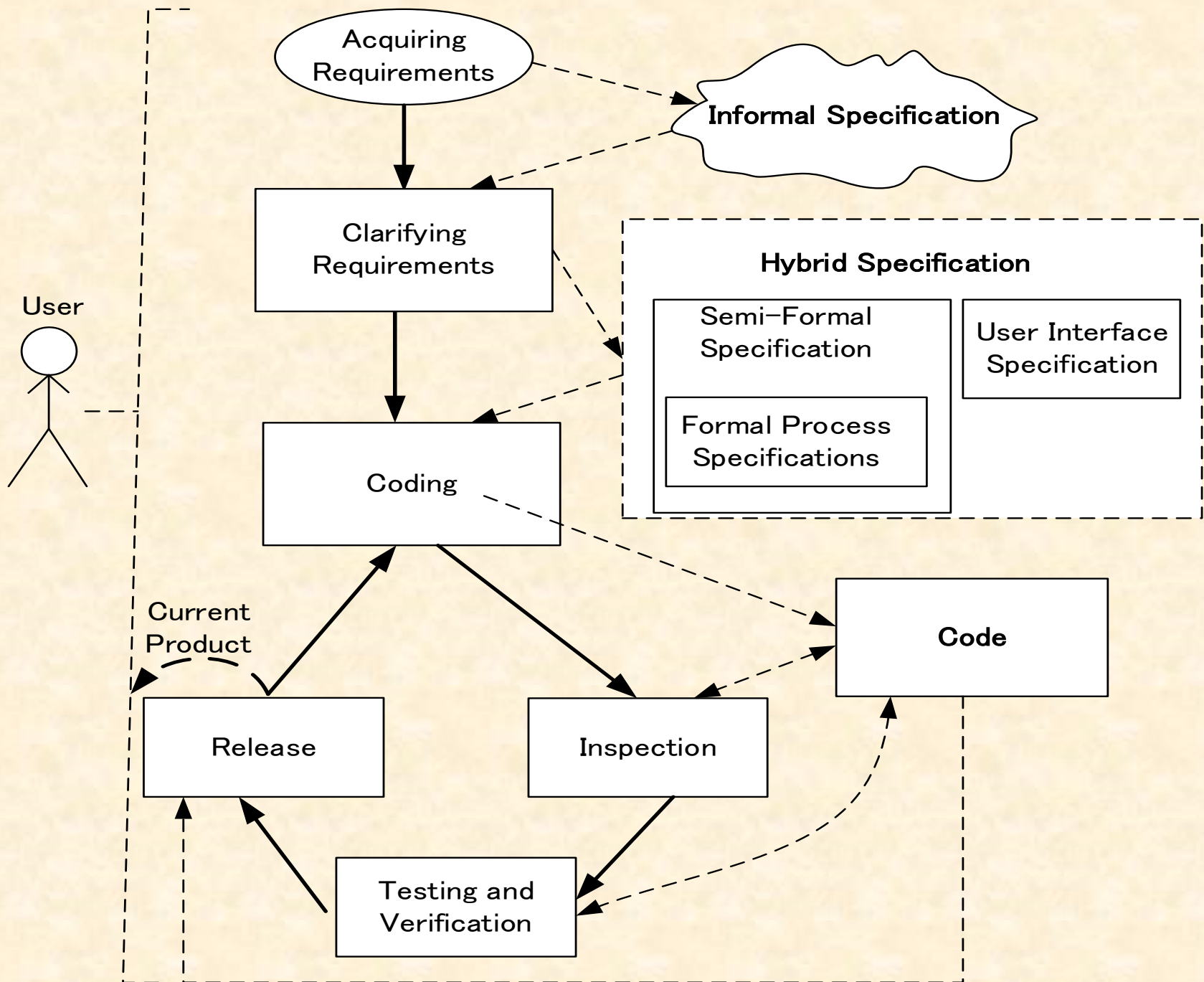


HP: <https://home.hiroshima-u.ac.jp/sliu/>

2.4 Agile-SOFL: Agile Formal Engineering Method

Characteristics:

- Hybrid specification (semi-formal specification + GUI specification + formal specification of critical operations) instead of formal specification
- Functional scenario form instead of pre-post for specifications
- Specification-based incremental implementation
- Specification-based testing and inspection
- Evolution instead of refinement



Informal specification of a simplified Stock Trading System (STS)

I. Functions

1. Register customer (name, age, bank account, address, email address, telephone number, and purchased stocks)
2. Register stock (name, price, unit for sale, limit for selling to one customer)
3. Purchase stock (the purchased stock details should be recorded in the customer's information and the payment should be made using the customer's bank account)
4. Sell stock (the money resulted from the sale of stock should be put into the customer's bank account)

II. Data

1. Customer list (F_1, F_3, F_4)
2. Stock list (F_2, F_3, F_4)
3. Bank accounts (F_3, F_4)

III. Constraints

1. For each kind of stock, a customer can only buy at most 1000 units (F_3, D_2).
2. A stock name is unique (D_2).
3. The currency used in the system is only CNY (F_3, F_4, D_2).

(1) Hybrid Specification

Hybrid specification =

Semi-formal specification +

GUI specification +

formal specification of critical operations

Semi-formal specification

```
process Register_Stock(stock: StockInformation) res: bool  
ext wr stock_list: seq of StockInformation
```

FSF:

The input **stock** does not exist in the **~stock_list** &&
the **stock_list** contains the **stock** and **res** = true

||

The input stock exists in the **~stock_list** &&
the **stock_list** is the same as **~stock_list** and **res** = false

||

others && **stock_list** = **~stock_list** and **res** = false

```
end_process
```

GUI specification

GUI views can be represented using PowerPoint software.

GUI elements:

- (1) message elements
- (2) data elements
- (3) functional elements
- (4) control elements

Welcome to the Stock Trading System

Message
element

Register Customer

Register Stock

Purchase Stock

Sell Stock

Functional
element

Provide the customer's information

ID:

Password:

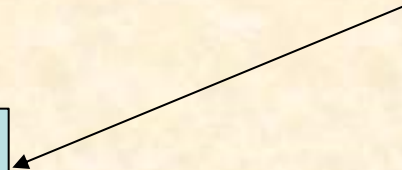
Name:

Telephone Number:

Email address:

Postal address:

Functional
element



Provide the customer's information

ID:

Password:

Name:

Telephone Number:

Email address:

Postal address:

`module` GUI;

`process` Register_Customer_Page1(Register Customer) Page 2

FSF:

Click on the button Register Customer &&
the GUI Page 1 is changed to Page 2.

comment

This process corresponds to the Register_Customer process specified in the module Registration.

`end_process`;

`process` Register_Stock_Page1(Register Stock) Page 3

FSF:

Click on the button Register Stock &&
the GUI Page 1 is changed to Page 3.

comment

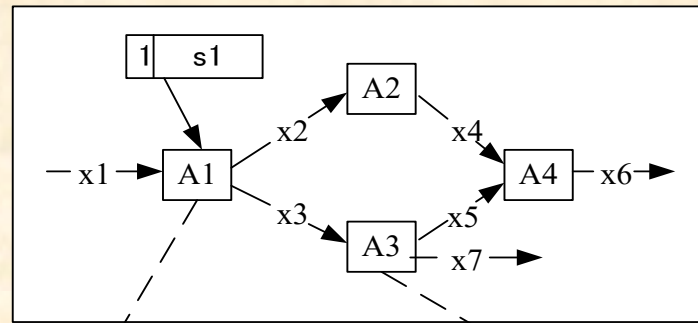
This process corresponds to the Register_Stock process specified in the module Registration.

`end_process`;

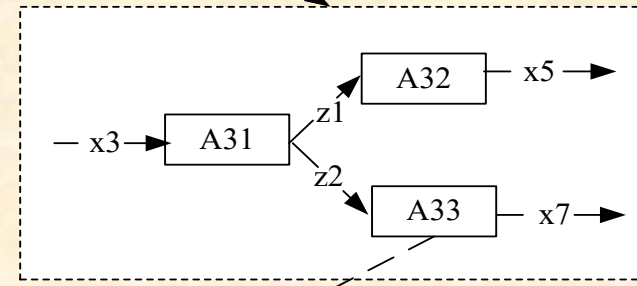
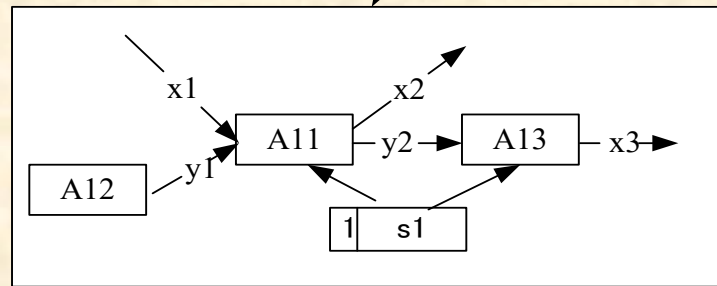
Formal specification

```
process Register_Stock(stock: StockInformation) res: bool
ext wr stock_list: seq of StockInformation
FSF:
stock notin elems(~stock_list) &&
stock_list = conc(~stock_list, [stock]) and res = true
||
stock inset elems(~stock¥_list) &&
stock_list = ~stock¥_list and res = false
||
others && stock_list = ~stock¥_list and res = false
end_process
```

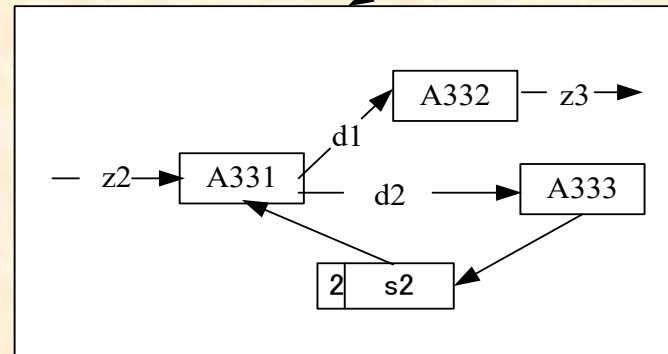
**The Whole
specification:
hierarchy of
modules.**



**Each module
contains a set
of process
specifications.**

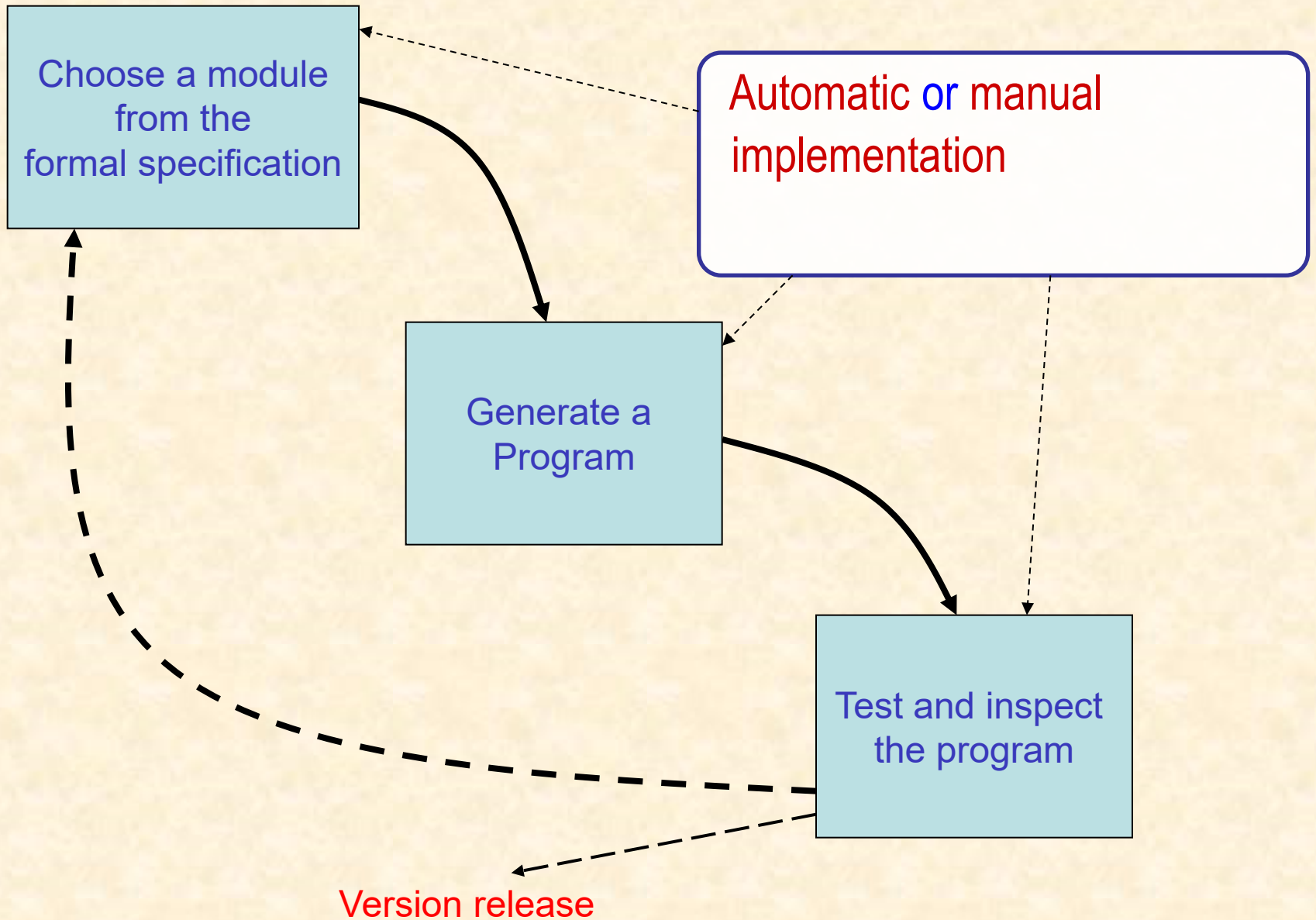


**Each process is
defined using FSF
specification.**

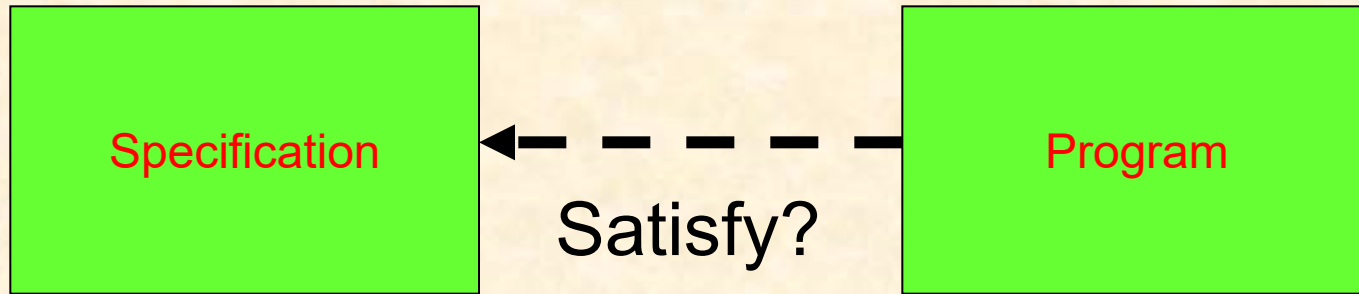


Specification-based incremental implementation

We take the bottom-up approach to automatically or manually (with tool support) implement and test the system based on the formal specification in an incremental fashion.



Testing-based testing and inspection



using specification-based testing and inspection

The characteristics:

Testing and inspection of the program are performed based on semi-formal specifications, formal specifications, and GUI specifications (semi-formal).

Evolution of programs

The research on HMPP focuses on the observer's role of computer and on dealing with the problem of how to construct a **correct program S** through a series of **evolutions of partial programs**:

$$S_1 \ll S_2 \ll \cdots \ll S_n = S$$

where each S_i ($i = 1..n$) is a **partial program** (program segment), and S is the **completed program**.

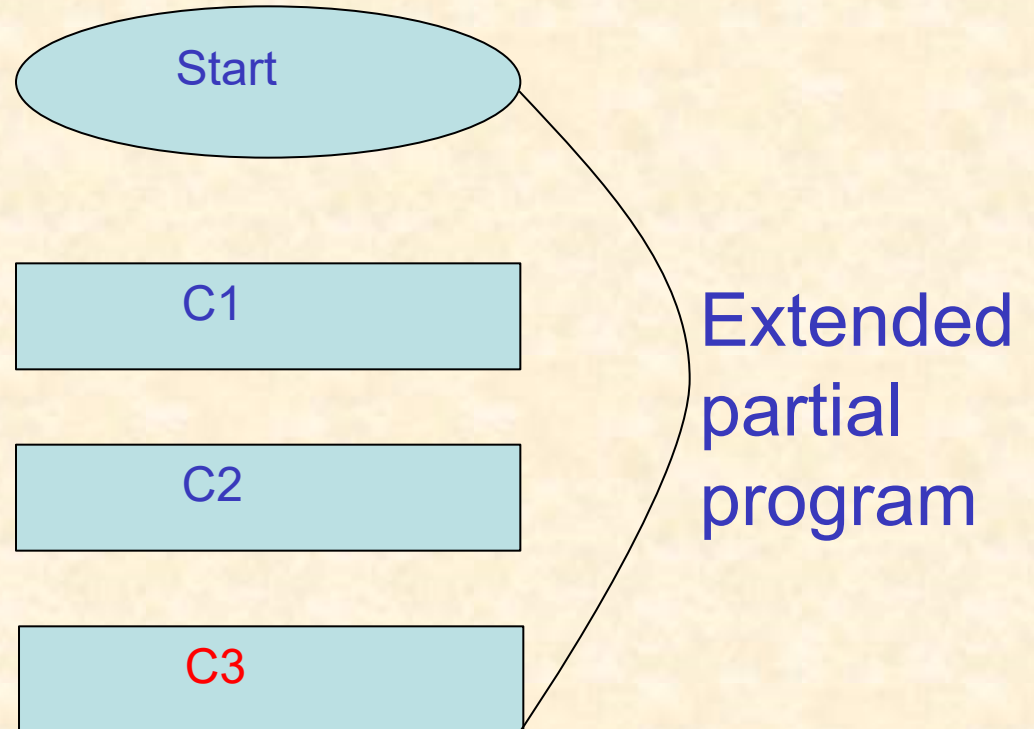
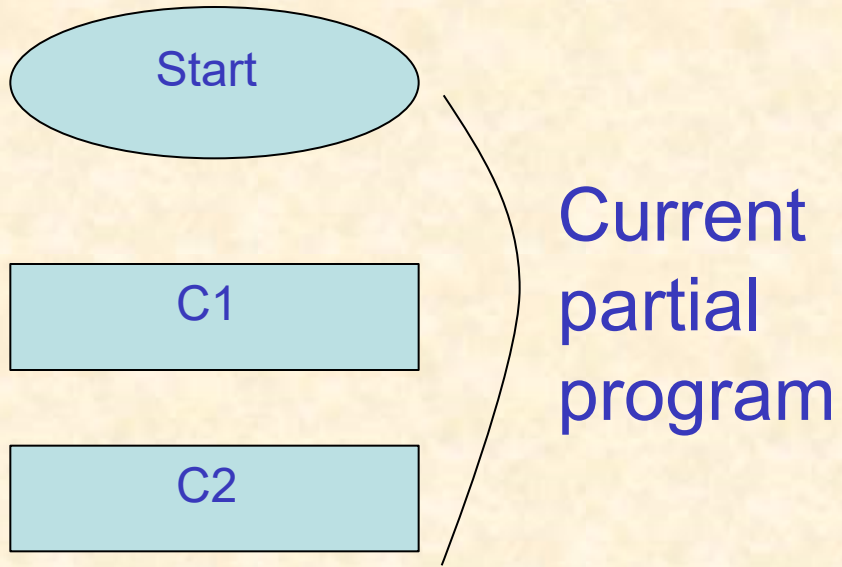
$S_1 \ll S_2$ means that S_1 is evolved to S_2 , or S_2 is an evolution of S_1 .

Definition of program Evolution

Definition: A partial program S is a sequence of commands, denoted by $S = [C1, C2, \dots, C_m]$, where each C_i ($i = 1, \dots, m$) is a command (a specification or code).

Definition: Let $S1 = [C1, C2, \dots, C_m]$ and $S2 = [C1', C2', \dots, C_n']$. Then, $S1$ is extended to $S2$, denoted by $S1 \leq S2$, if and only if they satisfy the condition:

$$C_i = C_i' \text{ (for } i = 1, \dots, m) \text{ and } n > m$$

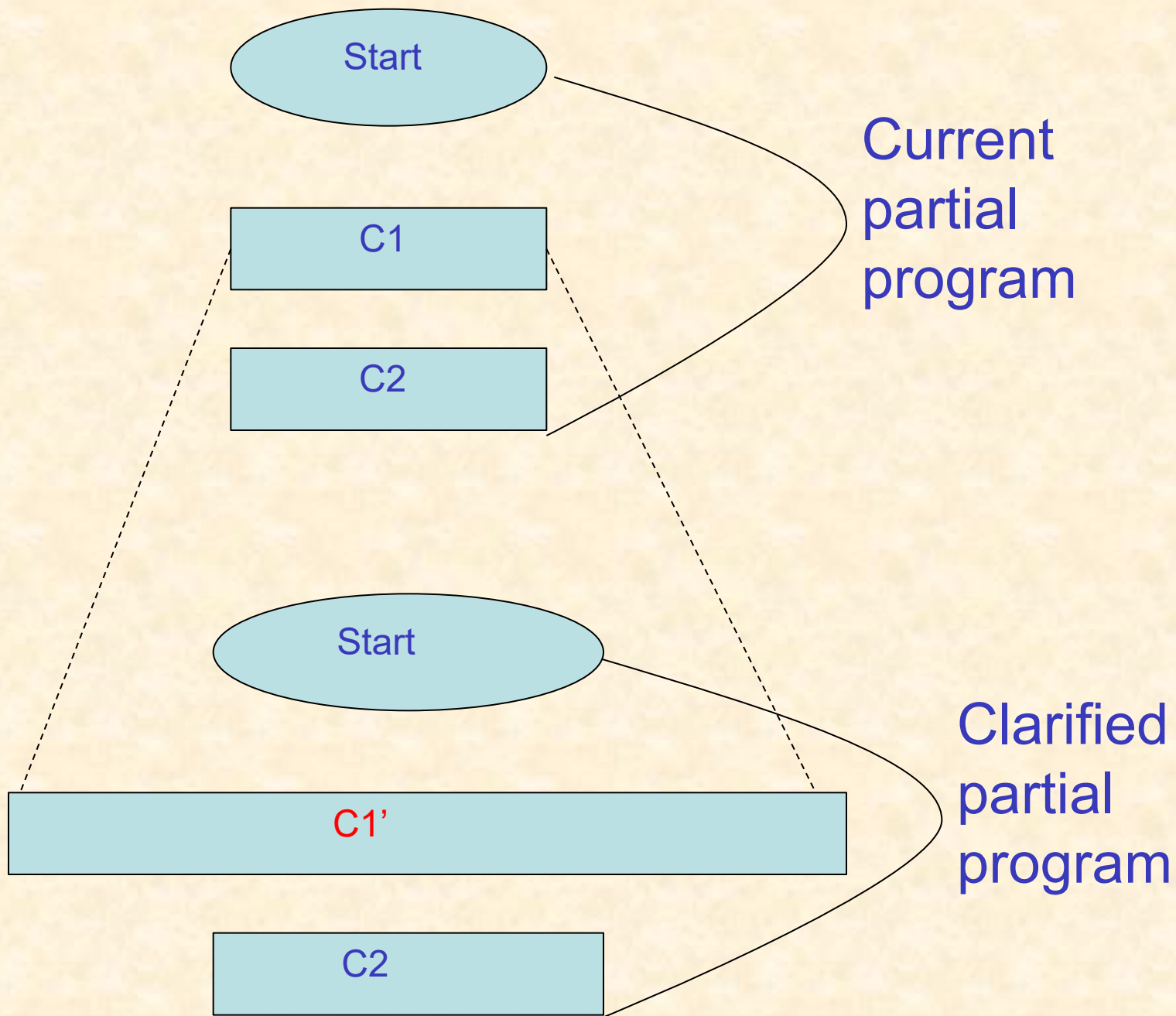


Definition: Let $S1 = [C1, C2, \dots, Cm]$ and $S2 = [C1', C2', \dots, Cn']$. Then, $S1$ is clarified to $S2$, denoted by $S1 \sqsubset S2$, if and only if they satisfy the condition:

$n = m$ and for some Ci ($1 \leq i \leq m$), Ci is redescrbed more clearly by Ci' .

In this case, we also say $S2$ is a clarification of $S1$.

The clarification also defines the human's responsibility.



Definition: Let $S1$ and $S2$ be two programs.
Then, $S1$ is *evolved* to $S2$, denoted by $S1 \ll S2$,
if and only if they satisfy the condition:

$S1$ is either extended to $S2$ or
 $S1$ is clarified to $S2$.

2.5 Testing-Based Formal Verification (TBFV)

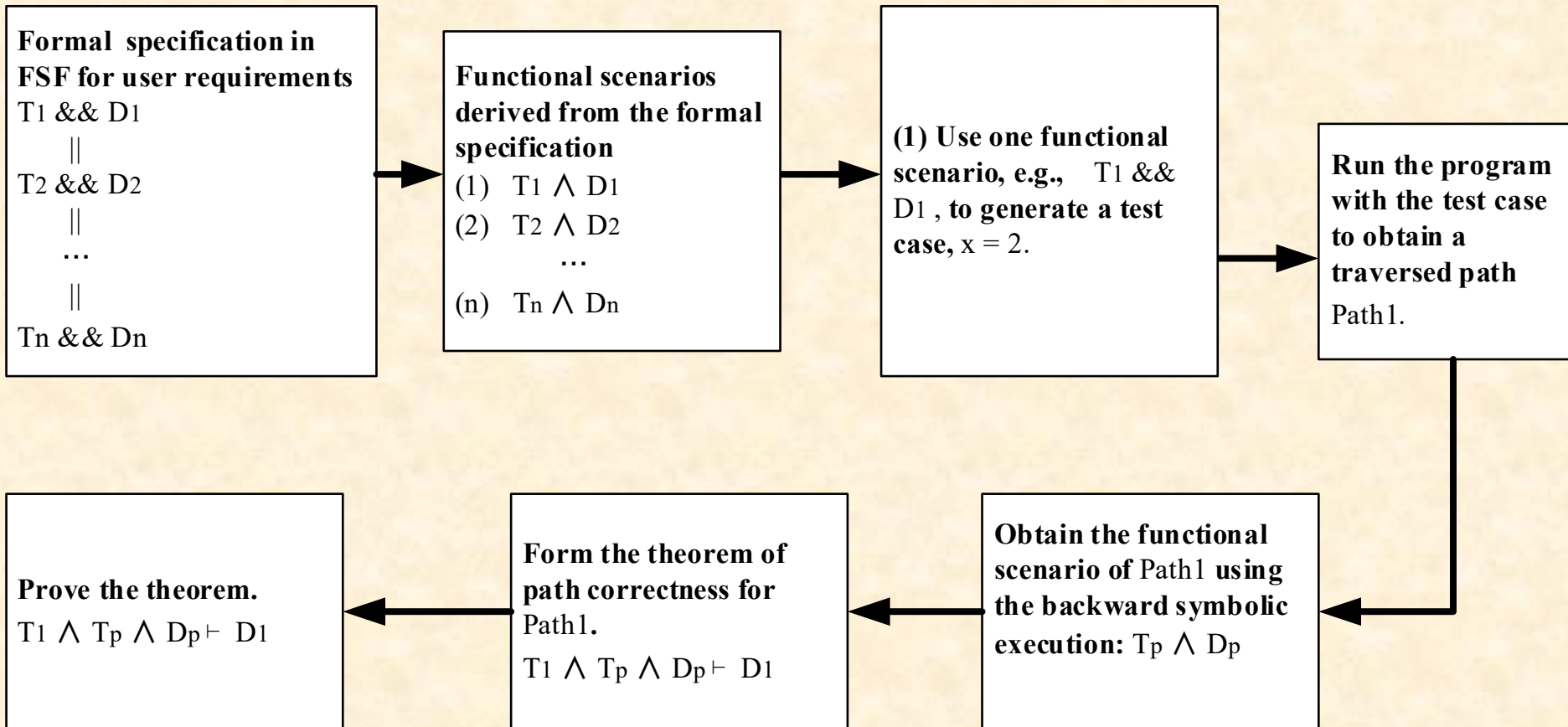
The principle of TBFV:

TBFV integrates specification-based testing and symbolic execution to ensure that a program is both tested for validation and formally verified for its correctness with respect to the specification.

The characteristics of TBFV:

A test case is generated from the specification and used to execute the program to obtain an execution path. Then, the correctness of the path is formally verified against the specification.

The procedure of TBFV



Example

Operation: Search for an element in a sequence.

Formal specification:

process Search(x: int, list: seq of int) pos: int

FSF:

exists[i: inds(list)] | list(i) = x &&

pos=hd([i | i: inds(list) & list(i) = x])

||

x notin elems(list) && pos = -1

end_process.

Functional scenarios (derived from the specification):

(1) $\text{exists}[i: \text{inds}(\text{list})] \mid \text{list}(i) = x \ \&\&$
 $\text{pos} = \text{hd}([i \mid i: \text{inds}(\text{list}) \ \& \ \text{list}(i) = x])$

(2) $x \text{ notin } \text{elems}(\text{list}) \ \&\& \ \text{pos} = -1$

Implemented program in Java:

```
int Search(int x, int list[]) {  
    int pos := -1;  
    int k := 0;  
    while (k < list.length) {  
        if (list[k] == x) {  
            pos := k;  
            break;  
        } else {  
            k := k + 1;  
        }  
    }  
    return pos;  
}
```

Test case generation and execution of code:

Test case: $x = 9$, $list = [3, 9, 20]$

(generated from functional scenario (1))

Traversed execution path:

```
pos := -1;
```

```
k := 0;
```

```
k < list.length
```

```
list[k] <> x;
```

```
k := k + 1;
```

```
k < list.length;
```

```
list[k] == x;
```

```
pos := k;
```

```
break;
```

```
return pos;
```

Backward symbolic execution to obtain the functional scenario for the path:

$\{0 < \text{list.length} \text{ and } \text{list}[0] \neq x \text{ and } 0+1 < \text{list.length} \text{ and } \text{list}[0+1] = x;$

$\text{pos}' = 0+1\}$

$\text{pos} := -1;$

$\{0 < \text{list.length} \text{ and } \text{list}[0] \neq x \text{ and } 0+1 < \text{list.length} \text{ and } \text{list}[0+1] = x;$

$\text{pos}' = 0+1\}$

$k := 0;$

$\{k < \text{list.length} \text{ and } \text{list}[k] \neq x \text{ and } k+1 < \text{list.length} \text{ and } \text{list}[k+1] = x;$

$\text{pos}' = k+1\}$

$k < \text{list.length}$

$\{\text{list}[k] \neq x \text{ and } k+1 < \text{list.length} \text{ and } \text{list}[k+1] = x; \text{pos}' = k+1\}$

$\text{list}[k] \neq x;$

$\{k+1 < \text{list.length} \text{ and } \text{list}[k+1] = x; \text{pos}' = k+1\}$

$k := k + 1;$

$\{k < \text{list.length} \text{ and } \text{list}[k] = x; \text{pos}' = k\}$

$k < \text{list.length};$

$\{\text{list}[k] = x; \text{pos}' = k\}$

$\text{list}[k] == x;$

$\{ ; \text{pos}' = k\}$

$\text{pos} := k;$

Obtain the functional scenario of the path:

$T_p : 0 < \text{list.length} \text{ and } \text{list}[0] \neq x \text{ and } 0+1 < \text{list.length} \text{ and } \text{list}[0+1] = x$

$D_p : \text{pos} = 0+1$

Formulate the theorem for correctness of the path:

$T \wedge T_p \wedge D_p \vdash D$

where the corresponding functional scenario in the specification is: $T \wedge D$.

Specific theorem (with required syntax changes and expression simplification):

$\text{exists}[i: \text{inds}(\text{list})] \mid \text{list}(i) = x \wedge$

$0 < \text{len}(\text{list}) \text{ and } \text{list}(0) \neq x \wedge$

$1 < \text{len}(\text{list}) \text{ and } \text{list}(1) = x \wedge$

$\text{pos} = 1 \vdash$

$\text{pos} = \text{hd}([i \mid i: \text{inds}(\text{list}) \ \& \ \text{list}(i) = x])$ (Valid!)

2.6 Human-Machine Pair Programming

- (1) Why Human-Machine Pair Programming (HMPP)
- (2) Theoretical Foundation and Framework for HMPP
- (3) Knowledge Classification

(1) Why Human-Machine Pair Programming

(1) What is pair programming (PP)

Definition: Pair programming is one of the techniques in the Extreme Programming **agile software development** method in which two programmers work together at one workstation.

Why Not Other Methods

Model-Driven Development (MDD)

Model (e.g., UML) → Code → Testing

時間消耗,
文書管理麻煩, 变化多,
同顧客的交流受限

Formal Methods (FM)

Specification → Code → Verification

Component-Based Development (CBD)

Components → Code → Testing

找到合适的
components
很难.

Why Are Agile Methods Popular?

- Consistent with the **nature of evolution** (changes) in software development (design-oriented)
- Timely and **comprehensible communication and interactions** between the developer and the customer
- One level **documentation (code)** for time saving

Is Agile Methods Perfect?

No! It is impossible!!!

- ◆ **Frequent changes of code**
- ◆ **Architecture-related errors**
- ◆ **Data structure-related errors**
- ◆ **Algorithm-related errors**

**Insufficient
time and
consideration
for
requirements
analysis and
design.**

How Can We Improve It ?

- **Specification-Based Programming** (Agile-SOFL for improving understanding of requirements and design)
- **Pair Programming** (for improving interactions and cooperation)

Characteristics of Pair Programming

1) Two programmers work together. One is called *driver* and the other is called *observer* or *navigator*.



2) The *driver* writes code, while the *observer* reviews each line of code as it is typed in.

3) The two programmers switch roles frequently.

Problems:

- ❖ There is a lack of clear principle to govern the process of working together by the two programmers
- ❖ The work can be considerably affected when the collaboration of the two programmers does not go well.



- ❖ It is more costly than one-person programming.

Question

How to **take advantage** of both **specification-based programming** and **pair programming** and **limit their disadvantages**?

Solution:

Human-Machine Pair Programming(HMPP) (for automatic monitoring, predicating, and incremental program review)

(2) What is human-machine pair programming (HMPP)?

Definition: **HMPP** means that a **programmer and computer work together** to construct a program, where the programmer plays the role of *driver* and the computer plays the role of *observer*.

program = specification or
code or
combination of both

Characteristics:

A) One programmer works together with computer in programming.



B) The programmer constructs the program, while the computer analyses the program that has been typed in to detect bugs and to predict code fragments.

C) It is much less costly than conventional pair programming and human mistakes will be significantly reduced.

(2) Theoretical Foundation and Framework for HMPP

The theoretical foundation defines the roles of computer in HMPP and the principle of fulfilling each role.

The framework provides an architecture and procedure for realizing the roles of computer (design for tool support) in HMPP.

An Evolutionary View of Programming

The research on HMPP focuses on the observer role of computer and on dealing with the problem of how to construct a **correct program S** through a series of **evolutions of partial programs**:

$$S_1 \ll S_2 \ll \cdots \ll S_n = S$$

where each S_i ($i = 1..n$) is a **partial program** (program segment), and S is the **completed program**.

$S_1 \ll S_2$ means that S_1 is evolved to S_2 , or S_2 is an evolution of S_1 .

(2.1) Theoretical Foundation

The roles of the computer observer:

1) Software Construction Monitoring (SCM)

1.1) Learning patterns for making faults from programming (where a fault is a syntactic expression that violates some property of the current partial program.)

1.2) Verifying properties of partial programs to detect potential faults

1.3) Reporting potential faults and the related information

2) Software Construction Predicting (SCP)

2.1) Self-correction of the mistakes in partial programs to remove faults

2.2) Predicting program fragments to enhance the robustness (or other properties, such as safety or security) of the program

2.3) Predicting program fragments towards completing the program

2.4) Reporting the predicted program fragments

3) Incremental Program Review (IPR)

3.1) Transforming programs to graphical representations for comprehension

3.2) Guiding the programmer to review the properly selected program fragments

3.3) Carrying out knowledge-based peer review

1.1) Learning patterns for making faults from programming

Analyse the process of editing the program to determine the patterns for making faults based on how many times the same expression or statement is repeatedly modified.

Example1: assume the decision

if ($x > y \ \&\& \ y < z$) {...}

is modified twice as follows:

$x > y \ || \ y < z \rightarrow x > z \ \& \ y < z \rightarrow x > y \ \&\& \ y < z$

Then, the computer will learn the pattern indicating that decisions containing the logical operator `||` is likely to contain mistakes.

Example2: assume that each of the three decisions

if ($x.age \geq 20$) { ...;}

...

if ($x > y \ \&\& \ x < z$) {...;}

...

while ($amount < balance$) {...;}

is modified twice during the editing of the program, then the computer will learn the pattern indicating that the decisions in conditional and iteration statements are likely to contain faults.

The same principle can be applied to other syntactic phenomena, such as variable declaration, function calls, assignments, nested statement, and components.

1.2) Verifying properties of partial programs to detect potential faults

Let S_1 be the current partial program. Let $P_1, P_2, \dots, P_n$ be properties S must satisfy.

SCM aims to automatically, dynamically check whether the current partial program S_1 satisfies these properties.

A property may be formed based on the specification or formed based on some implementation rules (e.g., prevent exceptions).

Challenges

- How to find and define all the properties $P_1, P_2, \dots, P_n$ that $S1$ must satisfy?
- Given a relevant property P_i ($i \in \{1, 2, \dots, n\}$), how can $S1$ be automatically and efficiently checked to determine whether it satisfies P_i or not?

Potential Techniques

For defining properties:

1. Specification-Based property definition
2. Exception-based property definition

For verifying the properties:

1. Specification-based static analysis
2. Predicate-based testing

1.3) Reporting potential faults and the related information

How to report the detected faults and potential faults will affect the accuracy and efficiency of human understanding and human-machine interactions.

The issues to address:

- (1) Format of the reported message (the structure of the message)
- (2) Presentation style of the reported message (the notation for expressing the message)
- (3) The level of the detail of the fault description

Challenges

- (1) How to ensure that all the reported faults are real faults for **S1** ?
- (2) How to ensure that the programmer will accurately and efficiently understand the reported faults given in the adopted format, style, and detail level?

2.1) Self-correction of the mistakes in partial programs to remove faults

After the programmer types in a line of program, the SCP system will automatically find some obvious mistakes and automatically correct them, respectively. After the correction, the results should be highlighted to remind the programmer of the changes.



The items that can be possibly self-corrected:

(1) Variable names

(2) Operators

(3) Function definitions

(4) Statements

(5) Decision and conditions

(6) Access restrictions (private, protected, public)

(7) Others

Possible techniques for self-correction:

(1) Change the syntax of the target

(e.g., F0rmal $\rightarrow$ Formal)

(2) Add necessary items to the target

(e.g., month ≥ 1 and month ≤ 12)

(3) Remove items from the target

(e.g., $a < 10 \ \&\& \ a < 20 \rightarrow a < 20$)

2.2) Predicting program fragments to enhance the robustness of the program

The robustness of a program is often concerned with input from the human-machine interface (e.g., GUI) and exceptions. The SCP system should automatically identify those parts and determine whether there is a need to improve the current partial program. If yes, then an appropriate program fragment should be added.

In general, the means for improving the robustness is **exception handling**. There might be other ways, such as adding **conditional statements** (e.g., a person's age must be greater than 0 and less than 150; the amount for withdrawal from ATM must be great then 0 and less than a specified limit).

2.3) Predicting program fragments towards completing the program

Suppose a completed program S is composed of n commands $C_1, C_2, \dots, C_n$. Abstractly, it is represented as $S = [C_1, C_2, \dots, C_n]$.

Let the current partial program be $CV_S = [C_1, C_2]$. Then predicting program fragments means to make an extension of CV_S , for example $CV_S' = [C_1, C_2, C_3, C_4]$.

A sequential program usually defines a mathematical function: given an input, it will produce an output.

When the current version of the program is written, the SCP system should automatically understand what should be written next and therefore propose a program fragment for this purpose.

Best  Best regards

Possible situations for proposing a program fragment:

- (1) Consider the logical expression (decision, condition) in **if-then-else** statements in order to ensure that for every possible situation, the corresponding processing statements are provided.

Example:

Current statement: if (amount <= balance && amount <= w_limit && amount > 0){ }

Proposed program fragment:

```
if (amount <= balance && amount <= w_limit && amount <= 0){  
...;}
```

```
if (amount <= balance && amount > w_limit && amount > 0){...; }
```

```
if (amount > balance && amount <= w_limit && amount > 0){...;}
```

(2) Consider an iteration statement to ensure that for every possible exit of the iteration statement, the corresponding processing statements are provided.

Example:

current statement: while (p1 || p2 || p3) { ...; }

proposed program fragment:

if (!p1) { ...; }

if (!p2) { ...; }

if (!p3) { ...; }

(3) Consider a class to ensure that the class contains all the necessary functions (or methods).

Example:

current class:

```
class account { string name;  
                string acc_no;  
                int balance;  
                int w_limit;  
                Deposit(int amount){...;}  
            }
```

proposed program fragment:

```
class account {...; //existing code  
                string getName(){return name;}  
                void setName(string na){name = na;}  
                ...  
                int Withdraw(?){?}  
                int Inquire(){?}  
            }
```

2.4) Reporting the predicted program fragments

How should the predicated program fragment be presented to ensure that the human programmer can accurately and efficiently understand it for deciding how it can be adopted in the current version of the program.

The issues to address:

- (1) Presentation style of the proposed program fragment (code, pseudocode, or diagram?)
- (2) The level of the detail of the proposed program fragment.

3.1) Transforming programs to graphical representations for comprehension

Examples:

- (a) Data flow diagrams
- (b) Control flow diagrams
- (c) Variable dependency graph

3.2) Guiding the programmer to review the properly selected program fragments

Activities:

(a) Select program fragments for review (e.g., where in the newly constructed program parts needs to be reviewed? Complexity? Importance?)

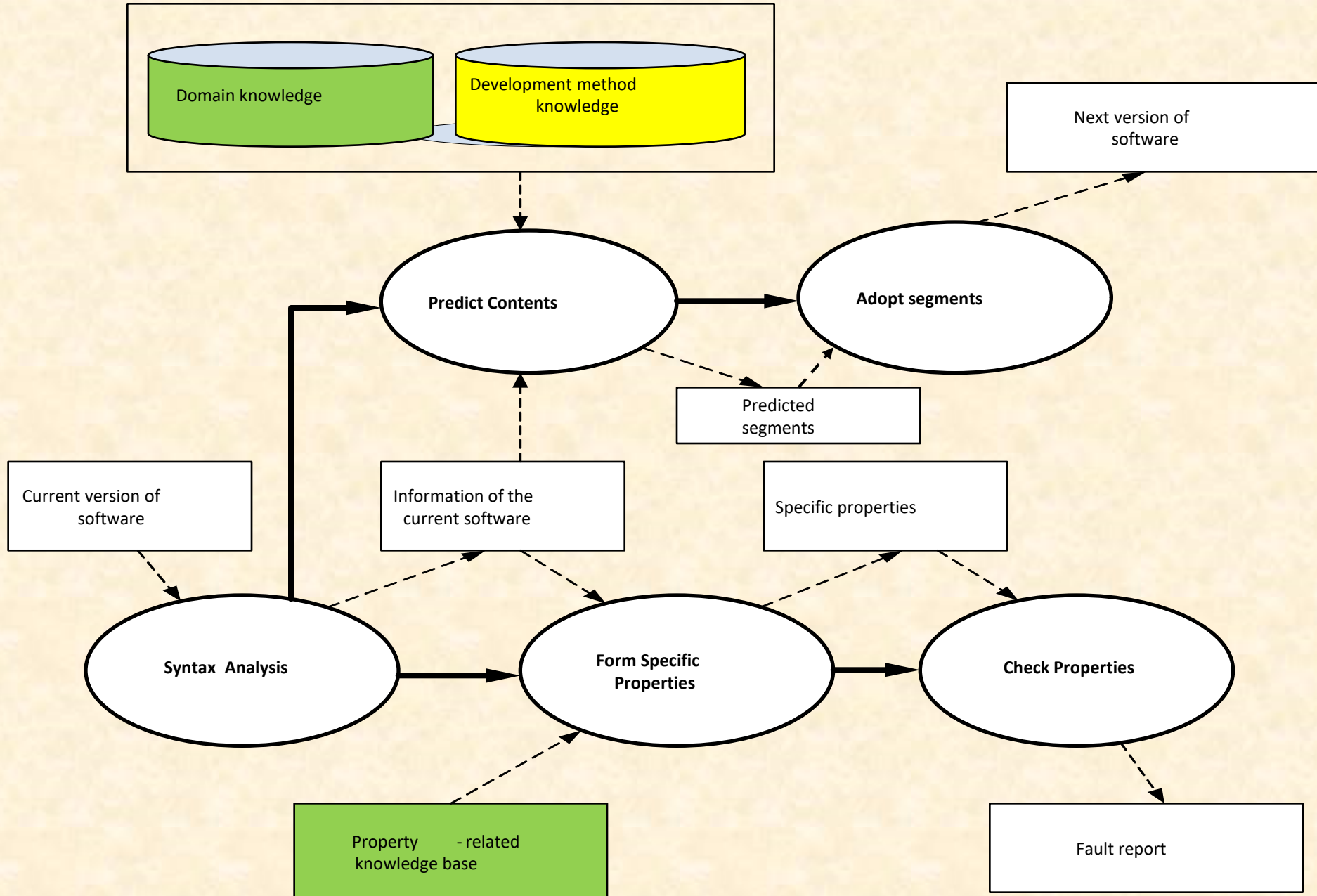
(b) Raise questions about the selected fragments to guide the review and support the review process

3.3) Carrying out knowledge-based peer review

Activities:

- (a) Build a knowledge base of bug patterns for programs on computer. Each bug pattern is a faulty expression, and it can be formed based on the domain, the specification, the exceptions, peers, and other sources.
- (b) Support the application of the knowledge to the current partial program.

Framework for HMPP



(3) Knowledge Classification

- (1) **Domain knowledge** (e.g., ATM, Railway card, Air ticket reservation, Railway control system)
- (2) **Method knowledge** (e.g., rules and procedures suggested by a specific programming method or software development method)
- (3) **Property-related knowledge** (e.g., various properties of variables, expressions, statements, modules)
- (4) **Fault-related knowledge** (e.g., common faults, specific faults)
- (5) **Standard-related knowledge** (e.g., the depth of nested conditional statements should be less than 5; variable declarations should be given together before the statements in the body of a function)
- (6) **Language-related knowledge** (e.g., a function can only return one value; no multiple inheritances of class is allowed)

(7) Self-correction knowledge (e.g., `class Derived::public Base` should be corrected to: `class Derived: public Base`)

(8) Robustness-related knowledge (e.g., input value should be converted into a consistent type; a person's age should be between 0 and 150 or 200)

(9) Extension knowledge (e.g., after the `while (p1 || p2 || p3) { ...;}`, there should be appropriate program statements to deal with the situations when each of the condition in the loop decision is false)

(10) Others

2.7 LLM-Based Software Engineering

Pretrained Large Language Models (LLMs) are the foundation of modern AI systems for tasks like text generation, translation, summarization, coding, and question answering.

Large – refers to the huge number of parameters (often billions or even trillions)

Language – refers to their ability to process natural languages such as English, Chinese, and Japanese.

Models – they are statistical models that learn patterns, structures, and meanings from data.

LLMs-Related Issues

Prompt Engineering is a research field that studies how to design and formulate instructions for Large Language Models (LLMs) in order to obtain satisfactory or desired responses.

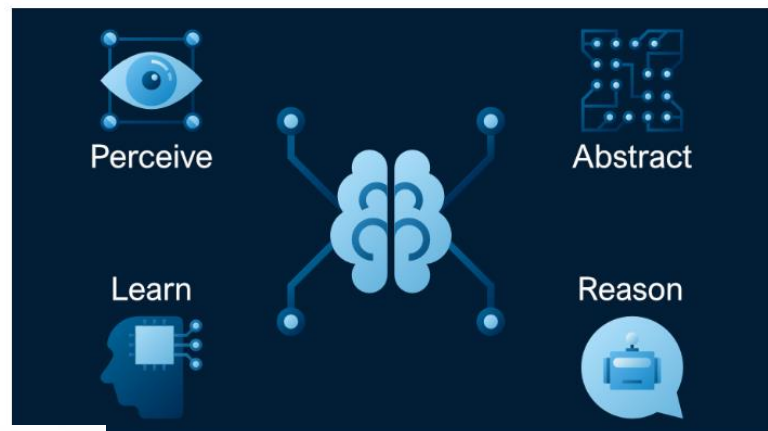
Challenge:

Nobody knows whether there is a best prompting method that can consistently lead to the most satisfactory responses from LLMs.

2.7.1 What is AI ?

Artificial Intelligence (AI) is an area of studying how to **simulate human intelligence by machines**, especially computer systems.

It includes **learning** (the acquisition of information and rules for using the information), **reasoning** (using rules to reach conclusions), and **self-correction**.



How can we know whether a machine has AI?

(1) **Alan Turing**: (born June 23, 1912, London, England—died June 7, 1954, Wilmslow, Cheshire). He entered the **University of Cambridge** to study mathematics in 1931, and **Princeton University** to study for a Ph.D. in mathematical logic under the American mathematical logician **Alonzo Church**'s direction (completed in 1938)



Alan Turing is a **British mathematician** and logician, who made major contributions to **mathematics**, **logic**, **philosophy**, and **mathematical biology** and also to the new areas later named **computer science**, **artificial intelligence**, and **artificial life**.

Well known contributions in computer science and AI:

- ◆ **Code breaker**
- ◆ **Turing Machine**
- ◆ **Artificial Intelligence (AI)**

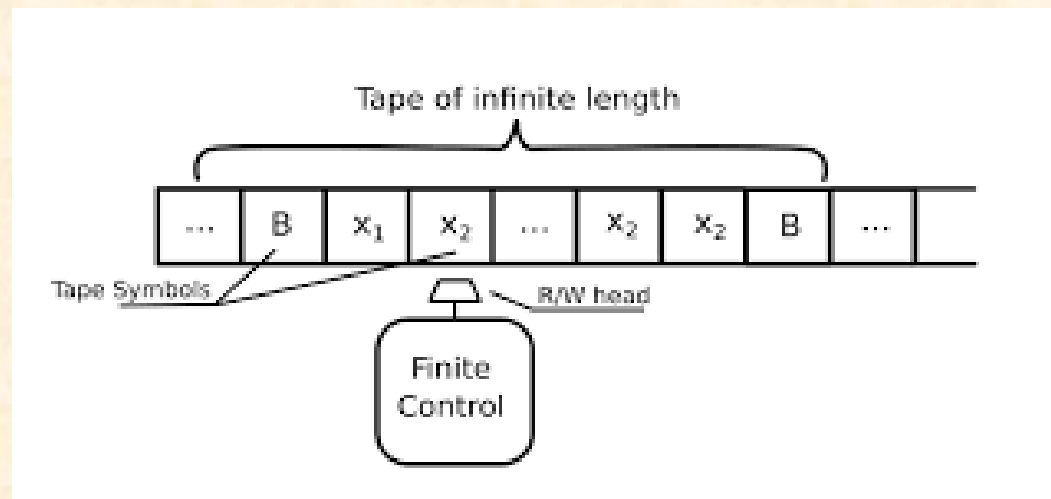
Code breaker

Turing was working for the British Government Code and Cypher School's wartime headquarters at Bletchley Park, Buckinghamshire during 1939 and 1945. He invented a code breaker called **Bombe** that can break the encrypted messages by **Enigma**, the principal cipher machine used by the German military to encrypt radio communications.



Turing Machine

Alan Turing invented the Turing Machine in 1936. It is not a machine in the ordinary sense but rather an idealized mathematical model based on which the first stored-program computer called Baby was built by the Royal Society Computing Machine Laboratory at the University of Manchester in June 1948. In 1949, the University of Cambridge developed a more powerful stored-program computer called EDSAC.



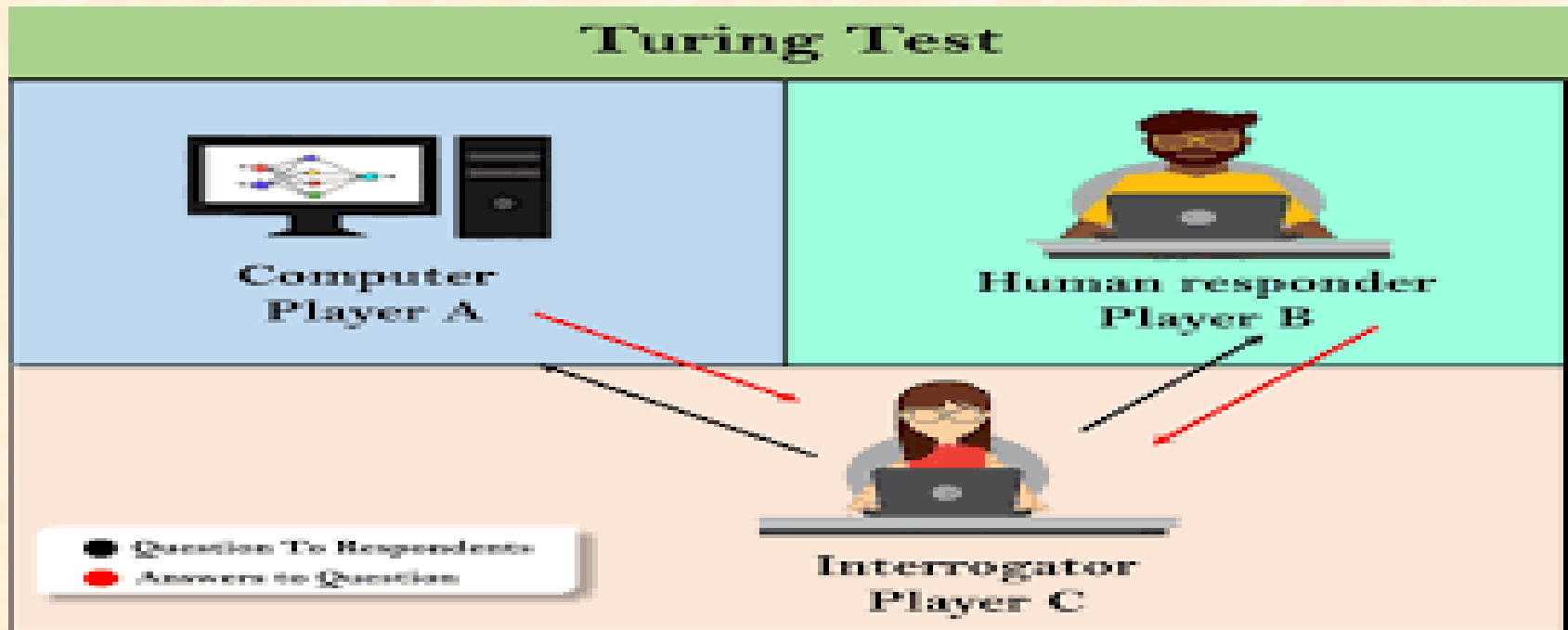
Artificial Intelligence (AI)

Turing was a founding father of artificial intelligence, and he was a leading early exponent of the hypothesis that the human brain is in large part a digital computing machine. He theorized that the cortex at birth is an “unorganised machine” that through “training” becomes organized “into a universal machine or something like it.”

Turing proposed what subsequently became known as the Turing Test as a criterion for whether an artificial computer is thinking (1950).

Turing Test

Turing test is a test to determine whether a computer can “think.” Specifically, the test is done in the following manner: **a remote human interrogator, within a fixed time frame, must distinguish between a computer and a human subject based on their replies to various questions posed by the interrogator.**



2.7.2 Specific fields in AI research and application

- Knowledge base and Expert Systems
- Computer Vision
- Pattern Recognition
- Natural Language Processing and Translation
- AI Computer game
- Robotics
- Machine Learning
- Deep Learning
- Human-Machine Pair Programming

2.7.2.1 Knowledge base and Expert Systems

A **knowledge base (KB)** is a technology used to **store** knowledge used by a computer system.

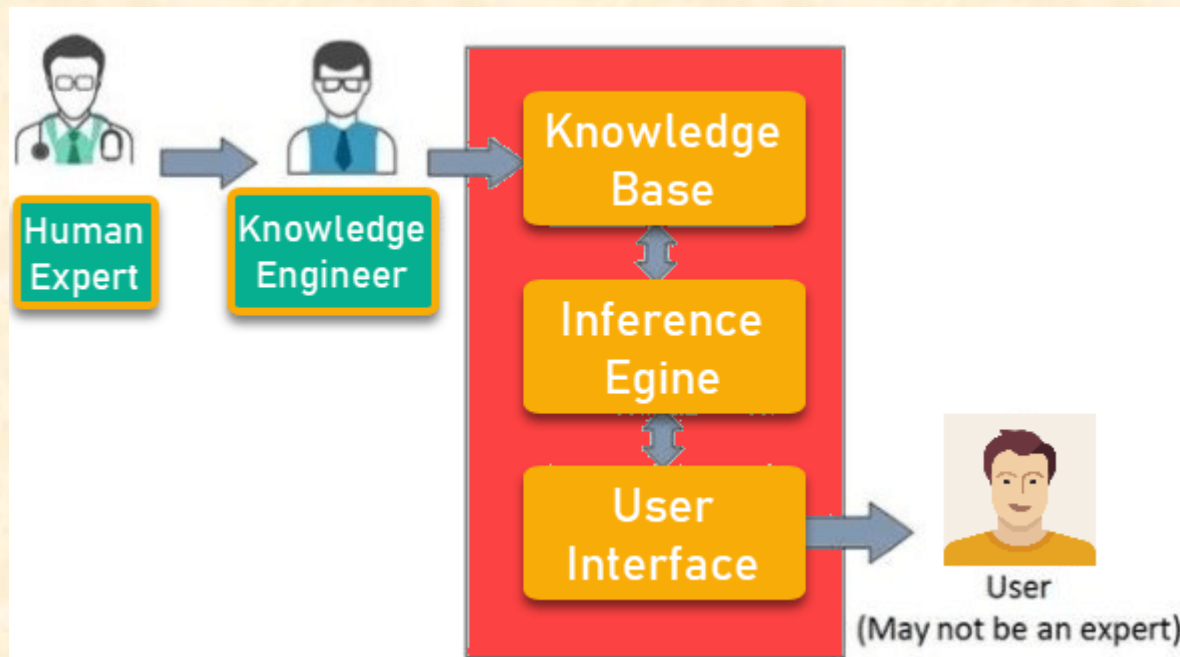
Knowledge is an inter-related **set of rules**. Each tells a conclusion under a given condition.

Examples of knowledge rules:

- (1) If a person is over 18 years old, he or she becomes an adult.
- (2) $1 + 7 > 8$ (if one does exercise for 1 hour and studies for 7 hours a day, the effect of the study will be greater than that of studying for 8 hours without exercise.)

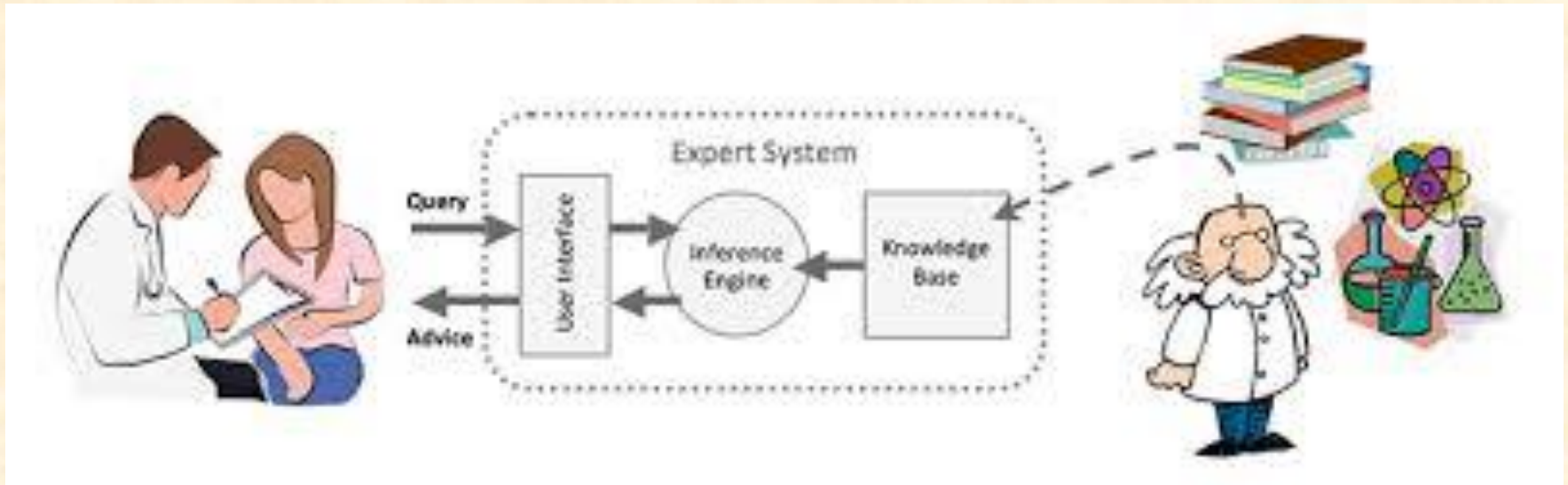
An **expert system** is a computer system that emulates the decision-making ability of a human expert. Expert systems are designed to solve complex problems by reasoning through bodies of knowledge, represented mainly as **if-then rules** rather than through conventional procedural code.

An **expert system** is divided into three subsystems: the **knowledge base**, the **inference engine** and the **user interface**. The knowledge base represents facts and rules. The inference engine applies the rules to the known facts to deduce new facts. The user interface takes the user's query in a readable form and passes it to the inference engine. After that, it displays the results to the user.



Examples:

(1) **Medical expert systems** (e.g., helping in diagnostic processes, laboratory analysis, treatment protocol, and teaching of medical students and residents.)



(2) **PROSPECTOR**: computer-based expert system, developed for the United States by the SRI's Artificial Intelligence Center to aid geologists in mineral exploration.



2.7.2.2 Computer Vision

Computer vision is the study of how computers can gain high-level **understanding from digital images or videos**. It seeks to understand and automate tasks that the human visual system can do.

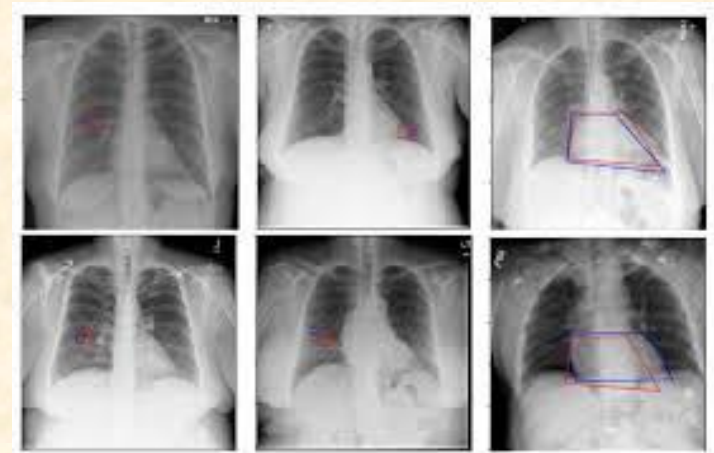
Computer vision tasks include methods for **acquiring, processing, analyzing and understanding digital images, and extraction of data** from the real world in order to produce numeric or symbolic information.

Applications of Computer Vision

◆ Autonomous driving systems: using computers to learn the road situation, make a decision, and control operations in road traffic for automobiles.



◆ Medical image analysis.



2.7.2.3 Pattern Recognition

Pattern recognition is the automated recognition of **patterns** in **data**.

A **pattern** is a **regularity** in the world, in human-made design, or in abstract ideas. The elements of a pattern repeat in a predictable manner.

Examples:

- Automated Facial Recognition
- Currency detector

2.7.2.4 Natural Language Processing and Translation

Natural language processing (NLP) deals with how computers can be used to process and analyze large amounts of natural language data.

Challenges in natural language processing frequently involve **speech recognition, natural language understanding, natural language generation, and natural language translation.**

Natural language translation is to study how one language can be properly translated into another language. This involves techniques for dealing with syntax, semantics, pragmatics, and synonyms of vocabulary.

2.7.2.5 AI Computer Game

AI computer game is a field to study how computer can act as a player of a game to play against another computer or human player.

Examples:

(1) **AlphaGo** is a computer program that plays the board game Go. It was developed by DeepMind Technologies which was later acquired by Google. In October 2015, the original **AlphaGo** became the **first computer Go program to beat a human professional Go player** without handicap on a full-sized 19×19 board.

2.7.2.6. Robotics

Robotics is an interdisciplinary research area concerned with the **design, construction, operation, and use of robots**.

A **robot** is a **machine**—especially one programmable by a computer— capable of carrying out a complex series of actions automatically.

Robotics utilizes the achievements of information engineering, computer engineering, mechanical engineering, electronic engineering and others.

Classification of Robots

- Industrial robots
- Construction robots.
- Agricultural robots
- Medical robots
- Robot combat for sport
- Domestic robots.
- AI robots

Examples of industrial robots



2.7.2.7 Machine Learning

Machine learning (ML) is the study of computer algorithms that improve automatically through experience.

In general, a machine learning algorithm works by **building a mathematical model** based on **sample data**, known as “**training data**”, in order to make predictions or decisions **without being explicitly programmed to do so**.

Examples of machine learning

機械学習技術の実世界応用

顔画像からの年齢認識

- 実験室と実環境での照明条件、カメラ設置条件、顔向きの違い、人種の違いなどを克服



非定常環境
適応学習

画像・動画処理

- 画像からの注目物体検出



- 動画からのイベント検出



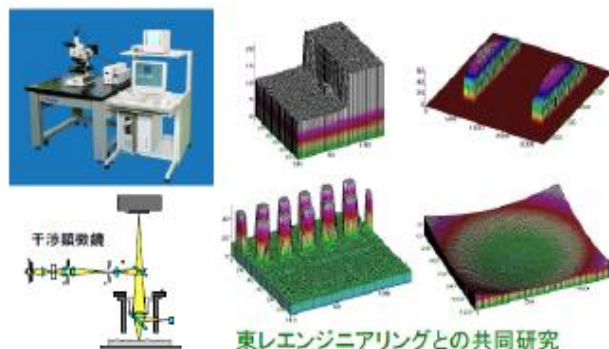
- 事前の学習は全く必要なく、与えられた画像や動画から即座に検出！

キヤノンとの共同研究

密度比推定

微細表面形状測定

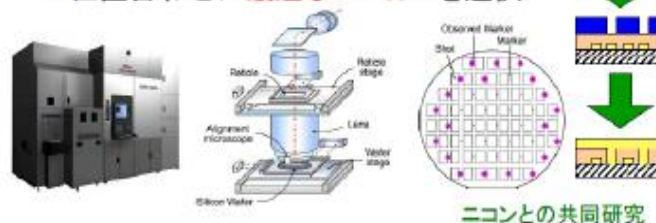
- ナノメートル単位の形状を光干渉を用いて一瞬で測定
- 局所的な類似性を用いて、高精度に推定



マルチタスク
学習

半導体露光装置における ウェハの位置合わせ

- 近年のシリコンウェハは多層構造を持つ
- 何度も回路パターンを露光するため、正確な位置合わせが非常に重要
- 位置合わせに最適なマーカーを選択



能動学習

2.7.2.8 Deep learning

Deep Learning is a subfield of machine learning concerned with algorithms inspired by the structure and function of the brain called **artificial neural networks**.

The **artificial neural networks** are built like the human brain, with neuron nodes connected together like a web.

While traditional programs build analysis with data in a linear way, the hierarchical function of deep learning systems enables machines to process data with a nonlinear approach.

The major difference between machine learning and deep learning

The machine learning algorithms build a model **based on a given correct solution** in the training data.

The deep learning algorithms build a model **without the need of a given correct solution** in the training data. This is possible due to the use of the artificial neural networks.

Example

We have a collection of **photos of dogs and cats**. Suppose we need to identify images of dogs and cats **separately** using machine learning algorithms and deep learning neural networks.

The **machine learning algorithm needs** a set of training data that clearly indicate which photo is dog and which one is cat, but the **deep learning neural networks do not need** such training data.

2.7.4 The future of AI

- (1) AI will continue to improve the efficiency and quality of our society.
- (2) AI systems will control our society and human activities to certain degree.
- (3) AI systems will become much better than human beings in some specific areas (e.g., speed, accuracy, memory), but will be extremely difficult to replace human beings due to the lack of creativity.

2.7.5 Roles of LLMs in Software Engineering

LLMs can be used to support almost every activity in software engineering. The following is the list of possibilities:

- ① Refine informal specifications to formal specifications.
- ② Code generation from both informal and formal specifications.
- ③ Perform code reviews and testing for bug detection.
- ④ Support formal verification of programs.

Challenges in using LLMs

Since there is no established method for prompting LLMs to produce the most satisfactory responses, their answers are often imperfect for human needs.

Question (for research):

How should we handle responses from LLMs, and what steps should we take to appropriately adopt them, either in whole or in part?

Questions ?